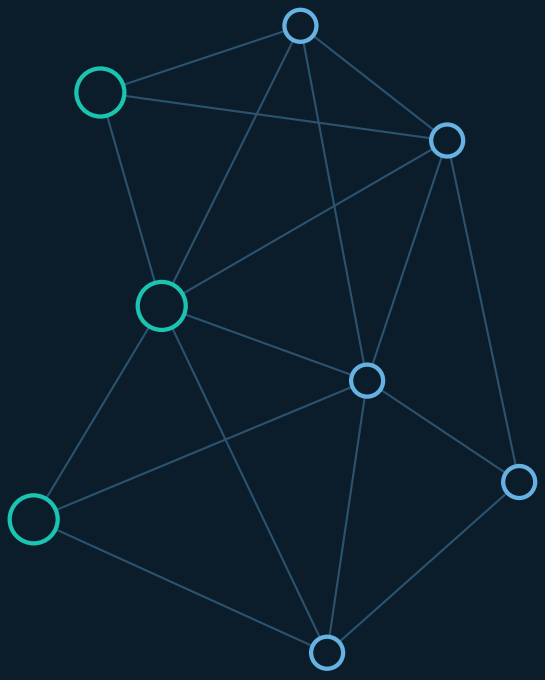
MODULE SCIENTIFIQUE • ÉDITION 2026

INTELLIGENCE ARTIFICIELLE

COURS THÉORIQUE ILLUSTRÉ

Comprendre les mécanismes. Vérifier les équations.
Expérimenter avant de conclure.



MATHÉMATIQUES ET DONNÉES

APPRENTISSAGE STATISTIQUE

RÉSEAUX DE NEURONES ET PYTORCH

NLP, ATTENTION, TRANSFORMERS ET LLM

RECHERCHE D'ÉTATS ET ROBOTIQUE

INGÉNIERIE ET FIABILITÉ DES SYSTÈMES

INCLUS DANS CE DOSSIER

16 FIGURES ORIGINALES • EXERCICES CORRIGÉS

SOURCES PRIORISÉES • SCRIPTS PYTHON

Complément au « Manuel IA — Édition Grands Défis 2026 »

Support de référence personnel · formation progressive en école d'ingénieurs

Sommaire détaillé

Avant-propos — Pourquoi un cours séparé ? **3**
Comment étudier chaque chapitre . . . 3
Une règle contre l'illusion de compréhension . . . 3
Bibliothèque minimale : ordre de priorité . . . 3
Carte conceptuelle . . . 4

Partie I — Mathématiques et programmation scientifique **5**
1. Vecteurs, matrices, tenseurs : le langage des données . . . 5
2. Dérivées, gradients et règle de chaîne . . . 6
3. Probabilités, statistiques et incertitude . . . 7
4. Python scientifique, NumPy, pandas et visualisation . . . 8

Partie II — Apprentissage statistique et rigueur expérimentale **10**
5. Qu'est-ce qu'un apprentissage automatique ? . . . 10
6. Régression linéaire, logistique et descente de gradient . . . 11
7. Jeux de données, validation, fuite de données . . . 12
8. Mesurer correctement une classification . . . 13
9. Modèles classiques : arbres, forêts, SVM et comparaison . . . 15

Partie III — Réseaux de neurones et PyTorch **16**
10. Neurone artificiel, couches et activations . . . 16
11. Backpropagation et optimisation des réseaux . . . 17
12. PyTorch : tenseurs, autograd et boucle d'entraînement . . . 18

Partie IV — Traitement du langage et modèles de langue **20**
13. Du texte brut aux vecteurs : NLP classique . . . 20
14. Embeddings, similarité et séquences . . . 21
15. Attention : comprendre précisément Q, K, V . . . 22
16. Architectures Transformers, pré-entraînement et LLM . . . 23
17. Construire et évaluer un mini modèle de langue . . . 24

Partie V — Recherche d'états, décision et Rubik's Cube **25**
18. Graphe d'états, permutations et problèmes de recherche . . . 25
19. BFS, DFS, IDDFS, A* et heuristiques . . . 25
20. Apprentissage par renforcement : extension, pas raccourci . . . 26

Partie VI — Fiabilité, ingénierie et grands défis **28**
21. Ingénierie expérimentale et reproductibilité . . . 28
22. Données réelles, déséquilibre, temps et changement de distribution . . . 28
23. RAG, évaluation des assistants et cybersécurité des systèmes d'IA . . . 29
24. Robotique, énergie et vision : quelles théories avant un grand défi ? . . . 30
25. Risques, limites et responsabilité de l'ingénieur IA . . . 30
26. Traduire la science en gestion de projet et leadership . . . 31

Partie VII — Exercices de synthèse et corrigés **33**

Partie VI bis — Compléments conceptuels pour aller plus loin **34**
Approfondissement A — Régularisation, convexité et stabilité numérique . . . 34
Approfondissement B — PCA, clustering et apprentissage non supervisé . . . 35

Approfondissement C — CNN, vision et segmentation 36
Approfondissement D — Séries temporelles et prévision de demande 36
Approfondissement E — D'une représentation à une décision sous contraintes 37
Approfondissement F — Transformer minimal : trajet des dimensions 38
27. Exercices corrigés, de la définition au calcul 39
28. Fiches de synthèse : ce que tu dois savoir expliquer sans notes 41
29. Sources complètes et ordre de lecture 42
30. Feuille de lecture progressive : quoi ouvrir, et quand ? 44

Avant-propos — Pourquoi un cours séparé ?

Ce support est le **cours théorique** complémentaire du *Manuel IA — Édition Grands Défis 2026*. Le manuel fixe les échéances et les livrables ; ce document explique les mécanismes que tu dois réellement comprendre. Il vise un étudiant d'école d'ingénieurs connaissant déjà la programmation et les mathématiques de classes préparatoires, mais découvrant l'apprentissage automatique. Il ne remplace pas intégralement les manuels universitaires cités ; il en ordonne l'étude et fournit des démonstrations pédagogiques originales.

Contrat scientifique. Distingue toujours (i) une définition mathématique, (ii) une intuition, (iii) une hypothèse du modèle et (iv) un résultat expérimental. Les figures de ce document sont générées spécifiquement pour ce cours à partir de données synthétiques. Ce ne sont ni des résultats de recherche, ni des figures empruntées à un article. Chacune porte une légende et des références sur lesquelles repose son explication.

Comment étudier chaque chapitre

1. **Avant lecture (5 min)** : réponds mentalement à « à quoi sert ce concept ? ». 2. **Lecture active (25–45 min)** : essaie de refaire chaque calcul, en notant les symboles. 3. **Expérience (20–60 min)** : exécute un script fourni, modifie une variable, explique l'écart. 4. **Restitution (10 min)** : écris une définition accessible à un lycéen, puis une définition précise avec formules. 5. **Réactivation** : reviens sur la notion à J+2 et J+7, et surtout au moment du projet correspondant.

Tu ne dois **pas** lire ce cours linéairement en quatre semaines. Le mois théorique couvre prioritairement les chapitres 1 à 12, puis des introductions guidées aux chapitres 13 à 19. Les chapitres spécialisés sur les Transformers, les solveurs et les risques deviennent centraux pendant les projets ou les grands défis. Considère ce document comme un **manuel de référence sur plusieurs mois** ; les 20–25 h par mois restent inchangées.

Une règle contre l'illusion de compréhension

Après chaque notion, produis trois preuves : une explication sans notes, un mini-calcul que tu vérifies à la main, et une expérience qui échoue si tu retires un élément essentiel. Exemple : pour la descente de gradient, explique le gradient, calcule une dérivée, puis fais volontairement diverger l'optimisation en choisissant un pas trop grand.

Bibliothèque minimale : ordre de priorité

P1 — Livre central, statistique et Machine Learning : Gareth James et al., *An Introduction to Statistical Learning, with Applications in Python* (2023), PDF officiel gratuit. Chapitres sur régression, classification, validation, arbres et apprentissage non supervisé. [R01]

P1 — Fondations mathématiques : Deisenroth, Faisal, Ong, *Mathematics for Machine Learning* (2020), PDF officiel accessible gratuitement. Étudie seulement les chapitres mathématiques pertinents lorsque tu bloques. [R02]

P1 — Deep learning, équations et code : Zhang et al., *Dive into Deep Learning*, livre interactif gratuit avec notebooks. [R03]

P1 — NLP / LLM : Jurafsky et Martin, *Speech and Language Processing*, manuscrit 2026 en ligne ; attention, il s'agit d'un **brouillon**. [R04]

P2 — Approfondissement conceptuel en Deep Learning : Simon J. D. Prince, *Understanding Deep Learning* (2023), en complément de [R03]. [R05]

P2 — Recherche d'états et IA classique : cours Berkeley CS188 + Russell et Norvig, *Artificial Intelligence: A Modern Approach* (livre commercial). [R06, R07]

P3 — Ingénierie avancée des modèles de langage : Stanford CS336, *Language Modeling from Scratch* (2026). Prérequis : PyTorch solide, optimisation et architecture Transformer. [R08]

Ces ressources ne sont pas à lire intégralement dans cet ordre : chaque chapitre ci-dessous donne **la section à cibler** pour limiter la dispersion. Les liens complets et leur nature (ouvrage, cours, documentation, article) figurent en fin de document.

Carte conceptuelle

- **Mathématiques et données** → vecteurs, probabilités, features et jeux de données.
- **Apprentissage statistique** → modèle, perte, optimisation, généralisation, évaluation.
- **Deep learning** → couches différentiables, backpropagation, PyTorch.
- **Langage** → tokens, TF–IDF, embeddings, Transformers et LLM.
- **Recherche / décision** → graphes, états, heuristiques et éventuellement reinforcement learning.
- **Ingénierie fiable** → expériences traçables, données sûres, tests, dérive et limites.

Partie I — Mathématiques et programmation scientifique

1. Vecteurs, matrices, tenseurs : le langage des données

À lire d'abord : Mathematics for Machine Learning (R02), chap. *Linear Algebra* ; [R09], *NumPy absolute basics*. **Ensuite :** [R03], chapitre sur les manipulations de données. **Prérequis :** opérations sur les réels et fonctions usuelles.

1.1 Du tableau Excel à l'espace vectoriel

Une **observation** peut être représentée par un vecteur $x \in \mathbb{R}^d$. Dans un problème de fraude, $x = (\text{montant}, \text{heure}, \text{ancienneté})$ peut représenter trois variables numériques. Pour n observations, on empile les lignes dans une **matrice de données** $X \in \mathbb{R}^{n \times d}$. La notation X_{ij} représente la valeur de la variable j pour l'observation i. Un **scalaire** est un nombre, un **vecteur** est une collection ordonnée, une **matrice** possède deux axes, un **tenseur** généralise cette représentation à plusieurs dimensions. En programmation, « tenseur » désigne souvent un tableau multidimensionnel ; en géométrie différentielle, le mot possède un sens plus profond que nous ne mobilisons pas ici.

Intuition : chaque observation est un point dans un espace de d dimensions. Un modèle cherche des régularités dans la position de ces points. Une image RGB de 32×32 pixels peut être vue comme un tableau $32 \times 32 \times 3$; un lot de 64 images devient $64 \times 32 \times 32 \times 3$ selon la convention d'axes utilisée.

1.2 Produit scalaire et produit matriciel

Pour deux vecteurs $u, v \in \mathbb{R}^d$, $u \cdot v = \sum_{j=1}^{d} u_j v_j$. Le produit scalaire mêle la longueur des vecteurs à leur orientation ; sous normalisation, il est relié à la similarité cosinus. Un modèle linéaire calcule $\hat{y} = w^\top x + b$. Pour traiter n lignes simultanément : $\hat{y} = Xw + b\mathbf{1}$, avec $X \in \mathbb{R}^{n \times d}$ et $w \in \mathbb{R}^d$. Les **dimensions doivent être compatibles** : une matrice (n, d) peut multiplier une matrice (d, k) pour produire une matrice (n, k).

Exemple calculé. Soit $x = (3, 5)$, $w = (2, -1)$, $b = 4$. Alors $w^\top x + b = 2 \times 3 - 1 \times 5 + 4 = 5$. Le modèle n'« comprend » pas le nombre 5 : il effectue une transformation numérique dont les coefficients devront être appris.

Une **norme** mesure la grandeur d'un vecteur. La norme euclidienne est $\|x\|_2 = \sqrt{\sum_j x_j^2}$. Une norme peut régulariser des paramètres pour limiter leur amplitude. Le produit de matrices et les normes reviennent dans les réseaux de neurones, la régression et les embeddings.

1.3 Broadcasting et dimensions

NumPy peut effectuer une opération entre des tableaux de formes différentes lorsque les dimensions sont compatibles selon ses règles de *broadcasting*. Par exemple un tableau $(100, 3)$ auquel on soustrait un vecteur $(3,)$: le vecteur est appliqué à chaque ligne. Le broadcasting n'est pas une

preuve que le calcul représente la bonne opération mathématique. Une dimension erronée peut produire un résultat **sans erreur de code**, mais scientifiquement faux.

```
import numpy as np
X = np.array([[1., 2.], [3., 4.]])  # forme (2, 2)
w = np.array([2., -1.])             # forme (2,)
b = 0.5
print(X @ w + b)                    # [0.5, 2.5]
print(X.shape, w.shape)
```

Piège : confondre Xw avec une multiplication terme à terme $X * w$; l'un agrège les caractéristiques, l'autre multiplie élément par élément, potentiellement avec broadcasting.

Vérifie-toi. (a) Une matrice $(80, 7)$ et un vecteur $(7,)$ produisent combien de prédictions ? (b) Une couche $7 \to 5$ requiert combien de poids hors biais ? (c) Pourquoi une image 28×28 représente-t-elle 784 intensités si on l'aplatit ?

2. Dérivées, gradients et règle de chaîne

À lire d'abord : Mathematics for Machine Learning (R02), *Vector Calculus* et *Continuous Optimization* ; [R03], section *Calculus*. **Ensuite :** [R10], Google ML Crash Course, *Gradient Descent*.

2.1 Une dérivée mesure une sensibilité locale

Pour une fonction d'une variable $f(x)$, la dérivée $f'(x)$ indique comment la fonction varie pour une petite variation de x. Géométriquement, c'est la pente de la tangente. Pour $f(x) = (x-3)^2$, on a $f'(x) = 2(x-3)$: pente négative à gauche de 3, nulle au minimum, positive à droite. Cette pente indique une direction locale de diminution de l'erreur.

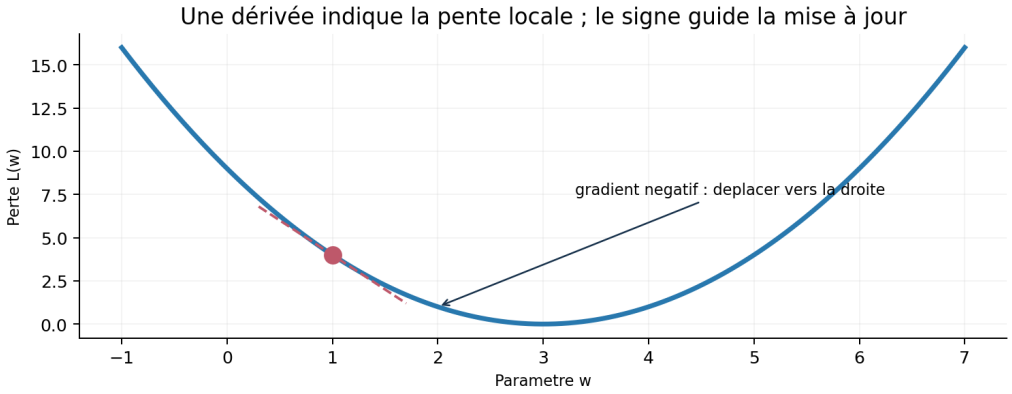


Figure 1: Pente locale, courbe quadratique et direction de mise à jour. Figure originale ; fondements : [R02, R10].

2.2 Plusieurs paramètres : le gradient

Lorsque la perte dépend de $w_1, \ldots, w_d$, on utilise les dérivées partielles. Le **gradient** $\nabla L(w)$ est le vecteur des d dérivées partielles $(\partial L/\partial w_1, \ldots, \partial L/\partial w_d)$. Dans un espace euclidien, il pointe dans la direction locale de croissance la plus rapide ; $-\nabla L$ est une direction de descente pour un pas suffisamment petit.

Pour $L(w,b) = (w-2)^2 + (b+1)^2$, le gradient vaut $(2(w-2), 2(b+1))$. Au point $(0,0)$, on obtient $(-4,2)$. Avec un pas $\eta = 0{,}1$, la nouvelle estimation est $(0,0) - 0{,}1(-4,2) = (0{,}4, -0{,}2)$. La perte initiale vaut 5 ; la nouvelle vaut $(1{,}6)^2 + (0{,}8)^2 = 3{,}2$.

2.3 La règle de chaîne, fondement de la rétropropagation

Si $z = f(x)$ et $y = g(z)$, alors $\frac{dy}{dx} = \frac{dy}{dz}\frac{dz}{dx}$. Exemple : $z = wx + b$, $L = (z-y)^2$; alors $\partial L/\partial w = 2(z-y)x$. Un réseau de neurones est une **composition de nombreuses fonctions**. La rétropropagation applique systématiquement cette règle de chaîne pour calculer les gradients de tous les paramètres en réutilisant les calculs intermédiaires.

Points de vigilance. Dérivée nulle ne veut pas toujours dire minimum global ; un minimum local peut ne pas être global ; un grand learning rate peut faire osciller ou diverger la fonction de coût. Les réseaux de neurones apprennent généralement avec des variantes stochastiques de ces calculs.

Exercice. Calcule $d((2w-3)^2)/dw$ puis évalue à $w = 1$. Vérifie avec une différence finie $[L(w+h) - L(w-h)]/(2h)$ pour $h = 10^{-4}$.

3. Probabilités, statistiques et incertitude

À lire d'abord : Mathematics for Machine Learning (R02), *Probability and Distributions* ; [R01], introduction à l'apprentissage statistique. **Ensuite :** [R03], *Probability and Statistics*.

3.1 Décrire les données

La **moyenne** $\bar{x} = \frac{1}{n}\sum_i x_i$ résume une tendance centrale ; la **médiane** est moins sensible aux valeurs extrêmes. La **variance** $s^2 = \frac{1}{n-1}\sum_i (x_i - \bar{x})^2$ (estimateur échantillonnal) mesure la dispersion quadratique ; l'écart-type s revient à l'unité initiale. Une **distribution** décrit la fréquence ou la probabilité des valeurs, et ne se réduit pas à sa moyenne. Pour les transactions bancaires, une distribution de montants peut être très asymétrique.

3.2 Probabilités conditionnelles et Bayes

$P(A \mid B) = P(A \cap B)/P(B)$ lorsque $P(B) > 0$. Le théorème de Bayes s'écrit $P(A \mid B) = \frac{P(B|A)P(A)}{P(B)}$. La distinction entre $P(\text{alerte} \mid \text{fraude})$ (sensibilité) et $P(\text{fraude} \mid \text{alerte})$ (probabilité qu'une alerte signale réellement une fraude) est fondamentale.

Exemple de taux de base : sur 10 000 transactions dont 1 % de fraude, un détecteur identifie 90 % des fraudes et émet à tort une alerte sur 5 % des opérations normales. On attend 90 vraies alertes et $0{,}05 \times 9\,900 = 495$ fausses alertes. La précision est $90/(90+495) \approx 15{,}4\%$. Même un détecteur très sensible peut produire une majorité de fausses alertes lorsque la fraude est rare.

3.3 Corrélation, causalité et échantillonnage

La corrélation linéaire mesure une association, pas un mécanisme causal. Deux variables corrélées peuvent l'être par l'effet d'une troisième variable. **Échantillonner** signifie choisir une partie d'une population. Un échantillon de commodité ou non représentatif peut introduire un biais de sélection. Dans le monde réel, les phénomènes changent dans le temps : la distribution observée pendant l'entraînement peut diverger de celle rencontrée ensuite.

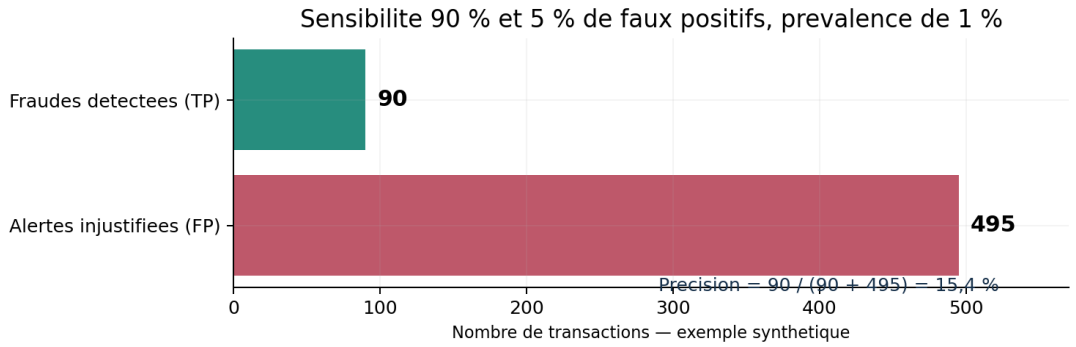


Figure 2: Effet du taux de base sur la signification d'un test. Schéma original construit sur un exemple numérique synthétique ; références [R01, R11].

Exercice. Recalcule la précision de l'exemple pour une prévalence de 0,1 %. Explique la modification **sans** changer la sensibilité et le taux de fausses alertes.

4. Python scientifique, NumPy, pandas et visualisation

À lire d'abord : NumPy — Guide officiel (R09), guide NumPy débutant ; [R12], tutoriels pandas. **Ensuite :** [R13], tutoriels Matplotlib. La programmation Python « classique » (fonctions, listes, dictionnaires, boucles, fichiers) est présupposée.

4.1 Pourquoi un tableau NumPy n'est pas une simple liste

`numpy.ndarray` stocke des éléments homogènes et permet des opérations vectorisées efficaces. Les propriétés `shape`, `ndim`, `dtype` déterminent ce que contient le tableau. Une **vue** peut partager sa mémoire avec le tableau d'origine : modifier une vue peut modifier les données sources. Une **copie** crée de nouvelles données. Cette distinction devient critique dans le nettoyage de données ou la manipulation d'images.

4.2 pandas : un modèle de données tabulaire

Un `DataFrame` associe des colonnes nommées, potentiellement de types différents. `isna()` compte les valeurs manquantes ; `groupby()` organise des agrégats ; `merge()` joint des tables. Avant toute modélisation, pose les questions : une ligne correspond à quoi ? une colonne a-t-elle une unité ? qui a saisi les données ? à quel moment cette information était-elle disponible ? Une colonne techniquement présente dans un fichier n'est **pas nécessairement autorisée comme entrée du modèle**.

```
import pandas as pd
transactions = pd.DataFrame({
    "type": ["TRANSFER", "PAYMENT", "TRANSFER"],
    "amount": [200., 12., 3400.],
    "fraud": [0, 0, 1]
})
print(transactions.groupby("type")["amount"].agg(["count", "mean"]))
print(transactions["fraud"].mean())  # prevalence, échantillon fictif
```

4.3 Visualiser pour poser de meilleures questions

Un **histogramme** montre une distribution, un **nuage de points** une relation, une **courbe** une évolution temporelle. Une matrice de corrélation peut générer des hypothèses mais n'établit jamais, seule, la causalité. Choisis les axes et unités honnêtement ; une échelle tronquée peut amplifier artificiellement les différences. Dans le code et le README, documente la source et la méthode de construction des figures.

Exercice. Crée un tableau de 100 montants fictifs dont cinq sont très élevés ; compare moyenne et médiane, puis trace l'histogramme en échelle linéaire et en échelle logarithmique.

Partie II — Apprentissage statistique et rigueur expérimentale

5. Qu'est-ce qu'un apprentissage automatique ?

À lire d'abord : An Introduction to Statistical Learning — Python (R01), *What is Statistical Learning?* ; [R10], Introduction ML. **Ensuite :** [R03], *Introduction*.

L'**intelligence artificielle** regroupe des méthodes permettant de réaliser des tâches demandant des capacités de perception, décision, prédiction, représentation ou recherche. Le **Machine Learning** apprend certains comportements à partir d'exemples, plutôt que de les écrire entièrement sous forme de règles. Le **Deep Learning** utilise souvent des réseaux de neurones à plusieurs couches pour apprendre des représentations. Ces trois notions ne sont pas interchangeables : une recherche A* sur un Rubik's Cube relève de l'IA, sans nécessairement mobiliser le ML.

5.1 Fonction d'hypothèse, perte, optimisation

Nous observons des exemples (x_i, y_i), où x_i est une entrée et y_i une cible. Nous choisissons une famille de fonctions f_θ, paramétrées par θ. L'entraînement vise à sélectionner des paramètres qui minimisent une **perte empirique** :

$$\widehat{R}(\theta) = \frac{1}{n} \sum_{i=1}^{n} \ell(f_\theta(x_i), y_i).$$

C'est une moyenne sur les exemples **disponibles**, pas une garantie de bonnes performances sur le monde réel. Le but est une **généralisation**, c'est-à-dire une faible erreur sur de nouvelles observations issues du problème qui nous intéresse.

Les **paramètres** (par exemple des poids w) sont appris à partir des données ; les **hyperparamètres** (par exemple la profondeur maximale d'un arbre ou le learning rate) sont fixés par une procédure de sélection et généralement évalués sur des données de validation.

5.2 Grandes familles d'apprentissage

- **Supervisé** : chaque observation a une cible. Régression (prédire un montant), classification binaire (fraude ou non), multiclasse (film, livre, électronique).
- **Non supervisé** : pas de cible explicitement fournie ; regroupement (*clustering*), réduction de dimension, détection de structure.
- **Auto-supervisé** : on construit la supervision à partir des données elles-mêmes (par exemple masquer ou prédire un token dans un texte).
- **Renforcement** : un agent choisit des actions dans un environnement, reçoit des récompenses et cherche à optimiser des retours cumulés.

Erreur fréquente : utiliser un modèle élaboré quand une règle simple ou un algorithme déterministe répond mieux au besoin. La bonne question est : « quelle information faut-il extraire, à quel coût et avec quel risque d'erreur ? »

6. Régression linéaire, logistique et descente de gradient

À lire d'abord : An Introduction to Statistical Learning — Python (R01), chapitres *Linear Regression* et *Classification* ; [R10], unités sur la régression et la classification. **Ensuite :** [R14], documentation des modèles linéaires scikit-learn.

6.1 Régression linéaire

Un modèle de régression simple prend $\hat{y} = wx + b$; avec plusieurs variables $\hat{y} = w^\top x + b$. Pour entraîner les paramètres, on peut minimiser la **MSE** (mean squared error) :

$$L(w, b) = \frac{1}{n} \sum_{i=1}^{n} (wx_i + b - y_i)^2.$$

Ses dérivées sont $\partial L/\partial w = \frac{2}{n} \sum_i (wx_i + b - y_i)x_i$ et $\partial L/\partial b = \frac{2}{n} \sum_i (wx_i + b - y_i)$. La **descente de gradient** effectue $w \leftarrow w - \eta\, \partial L/\partial w$ et $b \leftarrow b - \eta\, \partial L/\partial b$.

Exemple réduit. Un seul point $(x, y) = (2, 6)$ et un modèle initial $w = 1, b = 0$. La prédiction vaut 2 et la perte carrée 16. Le gradient de w vaut $2(2-6) \times 2 = -16$; pour $\eta = 0{,}1$, w devient 2,6. Sur un vrai jeu de données, on agrège les gradients sur plusieurs observations afin de choisir des paramètres qui ne se contentent pas de mémoriser une ligne.

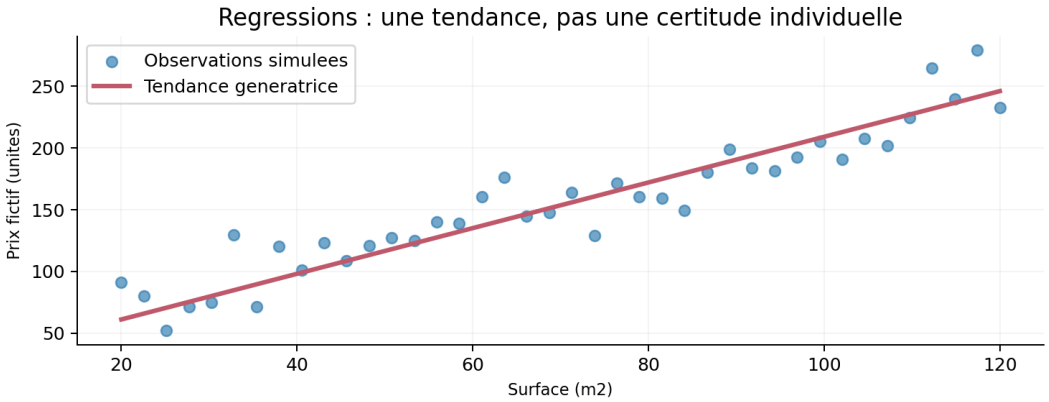


Figure 3: Une droite ajustée à des observations : exemples artificiels. Figure originale ; références [R01, R10].

6.2 Classification et régression logistique

Pour classer une transaction $y \in \{0, 1\}$, une sortie linéaire brute n'est pas une probabilité. On lui applique la **sigmoïde** $\sigma(z) = 1/(1 + e^{-z})$, d'où $p_\theta(y = 1 \mid x) = \sigma(w^\top x + b)$. Le modèle ne garantit pas, à lui seul, que les scores sont parfaitement **calibrés** ; cela se mesure.

La fonction de coût classique est la **cross-entropy binaire** :

$$L = -\frac{1}{n} \sum_i \left[y_i \log(p_i) + (1 - y_i) \log(1 - p_i) \right].$$

Intuition : attribuer une probabilité très faible à un événement réellement observé est lourdement pénalisé. Une **frontière de décision** apparaît lorsque le score traverse un seuil τ, souvent initialisé à

0,5 mais à choisir en fonction de la tâche et des coûts d'erreur.

6.3 Perte, précision et objectif métier ne sont pas identiques

La perte sert surtout à entraîner ; la précision (precision), le rappel ou le coût économique servent à évaluer et décider. Deux modèles peuvent présenter une perte proche mais des profils de faux positifs différents. Pour la fraude, la décision finale ne peut pas découler du seul score : il faut connaître le taux de fraude, les coûts d'investigation, les risques pour l'utilisateur et les conséquences d'un blocage.

Exercice. À $z = 0$, calcule $\sigma(z)$. Puis trouve qualitativement ce qui arrive à la probabilité lorsque $z \to \pm\infty$. Explique pourquoi on n'utilise pas la MSE par défaut pour entraîner une classification logistique.

7. Jeux de données, validation, fuite de données

À lire d'abord : scikit-learn — Common Pitfalls (R15), *Common pitfalls and recommended practices* de scikit-learn ; [R01], *Resampling Methods*. **Ensuite :** [R14], *Cross-validation*.

7.1 Trois jeux, trois fonctions distinctes

Le **train set** ajuste les paramètres, le **validation set** guide le choix des hyperparamètres et des seuils, le **test set** mesure la performance finale une fois les décisions fixées. Ne transforme pas le jeu de test en outil d'optimisation permanente : tu finirais par l'utiliser indirectement pour entraîner ton protocole expérimental.



Figure 4: Sous-ensemble d'entraînement, de validation et de test : proportions illustratives. Figure originale, méthodologie [R01, R15].

Un **split stratifié** conserve approximativement la proportion de classes dans les sous-ensembles. Pour les séries temporelles ou la fraude, un **split chronologique** peut être plus réaliste : entraîner sur le passé et tester sur le futur. Si plusieurs lignes concernent le même compte, utilisateur ou produit, un split **par groupe** évite des dépendances excessives entre train et test.

7.2 Le piège de la fuite de données (*data leakage*)

Une fuite survient lorsqu'une information indisponible au moment de la prédiction influence le modèle. Exemples : (a) utiliser le solde **après** un virement pour prédire s'il fallait accepter le virement ; (b) normaliser les variables à partir de la moyenne calculée sur train **et test** ; (c) faire figurer des transactions d'un même scénario quasi dupliqué dans train et test. Des scores extraordinaires peuvent alors masquer un modèle inutile en réalité.

Ordre correct : séparer d'abord les données ; ajuster les transformations sur le train seulement ; appliquer sans réajuster aux données de validation/test ; sélectionner les modèles sur validation ; tester une fois à la fin. La classe `Pipeline` de scikit-learn facilite cette discipline.

```
from sklearn.pipeline import make_pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.linear_model import LogisticRegression
pipeline = make_pipeline(StandardScaler(), LogisticRegression(max_iter=300))
pipeline.fit(X_train, y_train)  # le scaler apprend uniquement le train
pred = pipeline.predict(X_test)
```

7.3 Biais, variance et généralisation

L'**underfitting** correspond à un modèle trop contraint, qui ne rend même pas compte des données d'entraînement. L'**overfitting** correspond à un ajustement trop spécifique aux données vues, au détriment de nouvelles données. Des courbes d'apprentissage, la validation croisée, la régularisation et la collecte de données mieux représentatives aident à diagnostiquer ces situations. Le compromis biais–variance donne une intuition importante mais n'explique pas à lui seul la généralisation des grands réseaux modernes.

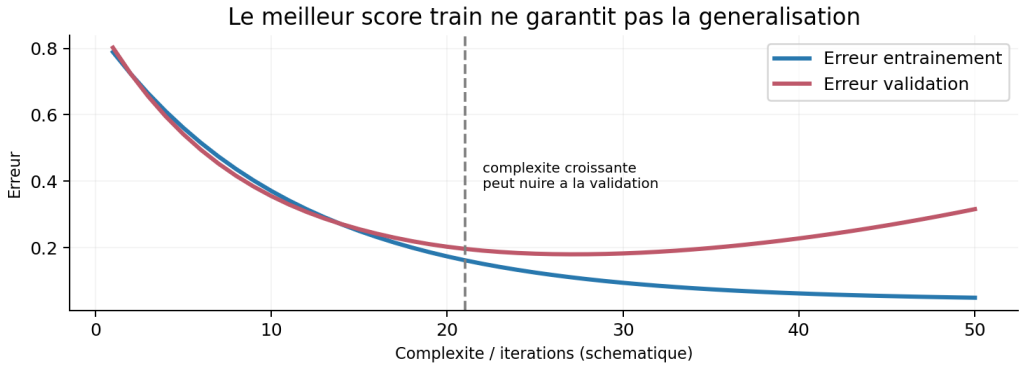


Figure 5: Écart schématique entre erreur d'entraînement et de validation. Figure originale illustrative ; sources [R01, R15].

Exercice. Quatre utilisateurs ont chacun dix transactions. Pourquoi un split aléatoire ligne par ligne pourrait-il être trompeur si le modèle contient l'identité ou des caractéristiques très spécifiques de l'utilisateur ?

8. Mesurer correctement une classification

À lire d'abord : scikit-learn — Model Evaluation (R11), documentation *Model Evaluation* ; [R01], *Classification*. **Ensuite :** [R16], *Precision-Recall*.

8.1 La matrice de confusion

Pour une classe positive « fraude », les **TP** sont les fraudes détectées, les **FN** les fraudes manquées, les **FP** les opérations saines faussement signalées, les **TN** les opérations saines laissées intactes. Une **matrice de confusion** rassemble ces quatre comptes. La convention des axes varie d'une bibliothèque à l'autre : toujours légender « réel » et « prédit ».

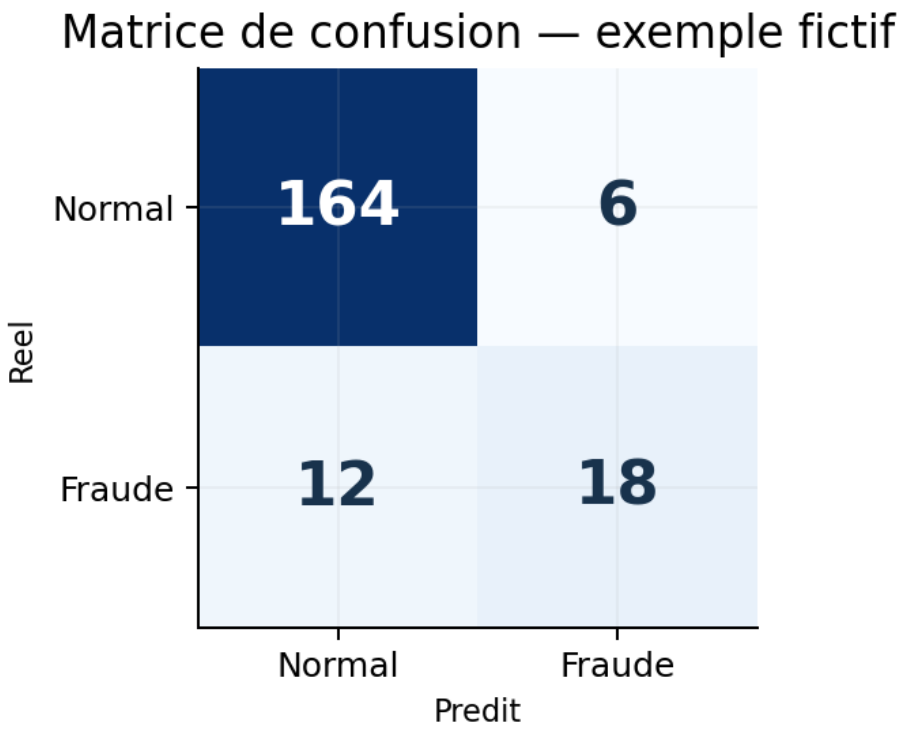


Figure 6: Matrice de confusion sur une expérience synthétique : les comptes sont fictifs. Figure originale ; référence [R11].

$$\text{accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

$$\text{precision} = \frac{TP}{TP + FP}, \quad \text{recall} = \frac{TP}{TP + FN}$$

$$F_1 = 2\frac{\text{precision recall}}{\text{precision} + \text{recall}}.$$

Exemple. Sur 100 transactions, 5 sont frauduleuses. Prédire « normal » sur toutes les lignes donne 95 % d'accuracy, mais **zéro fraude détectée**. Voilà pourquoi l'accuracy seule peut être une mauvaise métrique. La convention logicielle utilisée pour les divisions de dénominateur nul doit être explicitée.

8.2 Seuils, courbes PR, calibration

Abaisser un seuil permet généralement de détecter davantage de cas positifs, avec davantage de fausses alertes. La **courbe precision–recall** examine ce compromis sur plusieurs seuils ; l'aire résumée par *Average Precision* est utile sur classes rares, mais il faut toujours analyser les valeurs concrètes selon le cas d'usage. La **calibration** compare une probabilité prédite de, par exemple, 0,7 avec la fréquence observée parmi les cas recevant ce score. Un score de discrimination élevé n'implique pas une bonne calibration.

8.3 Coût de l'erreur et problèmes multiclasse

Pour la fraude, un faux négatif peut générer une perte financière, un faux positif une friction ou un blocage injustifié. Modélise explicitement les coûts lorsque c'est possible. En multiclasse, calcule une

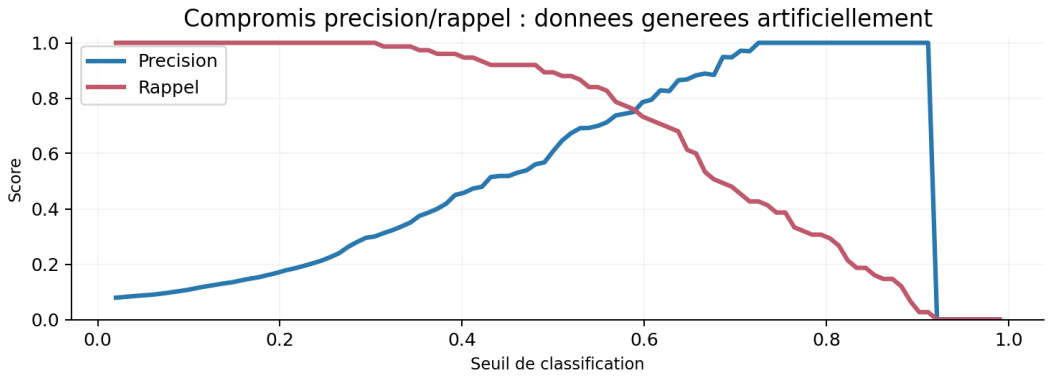


Figure 7: Précision et rappel selon le seuil : illustration sur données synthétiques, non issue d'un benchmark. Références [R11, R16].

matrice de confusion $K \times K$ et des métriques **macro** (moyenne donnant le même poids à chaque classe), **micro** (agrégation globale des décisions) ou **pondérées** (par support). Un modèle excellent sur la classe majoritaire mais mauvais sur les classes rares peut avoir un micro-score flatteur.

Exercice. Avec TP=18, FP=6, FN=12, TN=164, calcule précision, rappel et F1. Quel indicateur choisirais-tu si manquer un cas est dix fois plus coûteux qu'une alerte ?

9. Modèles classiques : arbres, forêts, SVM et comparaison

À lire d'abord : An Introduction to Statistical Learning — Python (R01), chapitres *Tree-Based Methods* et *Support Vector Machines*. **Ensuite :** [R14], documentation scikit-learn.

Un **arbre de décision** divise successivement l'espace des caractéristiques par des règles telles que « montant > 1000 » ou « ancienneté < 30 jours ». Il produit des régions de prédiction faciles à inspecter, mais un arbre profond peut très facilement mémoriser son échantillon. Une **Random Forest** combine de nombreux arbres entraînés sur des sous-échantillons et avec une sélection aléatoire de variables, réduisant souvent la variance d'un arbre isolé. Les méthodes de **gradient boosting** construisent au contraire une succession de modèles corrigeant progressivement des erreurs ; elles constituent souvent de bons candidats pour les données tabulaires structurées.

Une **SVM** linéaire cherche une frontière séparatrice à grande marge ; avec un **noyau** (*kernel*), elle peut représenter des séparations non linéaires. Le choix du modèle doit aussi considérer taille des données, temps, explicabilité, calibration, coût d'inférence et disponibilité d'une stratégie de validation pertinente.

Baseline : avant un modèle complexe, calcule toujours les performances d'une règle naïve puis d'un modèle simple (par exemple régression logistique). Un gain de 0,5 point de score n'est pas automatiquement intéressant si le modèle devient dix fois plus lent, opaque et difficile à maintenir.

Exercice. Propose une raison pour laquelle une Random Forest pourrait surpasser une régression linéaire sur la fraude ; propose également une raison pour laquelle tu choisirais malgré tout la régression logistique.

Partie III — Réseaux de neurones et PyTorch

10. Neurone artificiel, couches et activations

À lire d'abord : Dive into Deep Learning (R03), *Multilayer Perceptrons* ; [R05], premiers chapitres sur les réseaux feed-forward. **Ensuite :** [R17], PyTorch *Build the Neural Network*.

Un neurone artificiel calcule un **score** $z = w^\top x + b$, puis une **activation** $a = \phi(z)$. Une couche dense complète s'écrit $h = \phi(Wx + b)$. Si $x \in \mathbb{R}^d$ et que la couche a m neurones, $W \in \mathbb{R}^{m \times d}$ et $b \in \mathbb{R}^m$; il y a $md + m$ paramètres. Plusieurs couches composées peuvent approximer des fonctions non linéaires complexes, mais la taille du réseau ne garantit ni bonne généralisation ni compréhension causale du phénomène.

10.1 Pourquoi les activations sont indispensables

Si on empile deux couches **sans activation non linéaire**, $h = W_1 x + b_1$ puis $y = W_2 h + b_2$, alors $y = (W_2 W_1)x + (W_2 b_1 + b_2)$: l'ensemble se réduit à une seule transformation affine. Une **activation** comme ReLU, $\mathrm{ReLU}(z) = \max(0, z)$, introduit une non-linéarité. La sigmoïde produit des sorties entre 0 et 1 ; la **softmax** transforme une liste de scores z_k en une distribution $p_k = e^{z_k} / \sum_j e^{z_j}$, utilisée notamment en classification multiclasse.

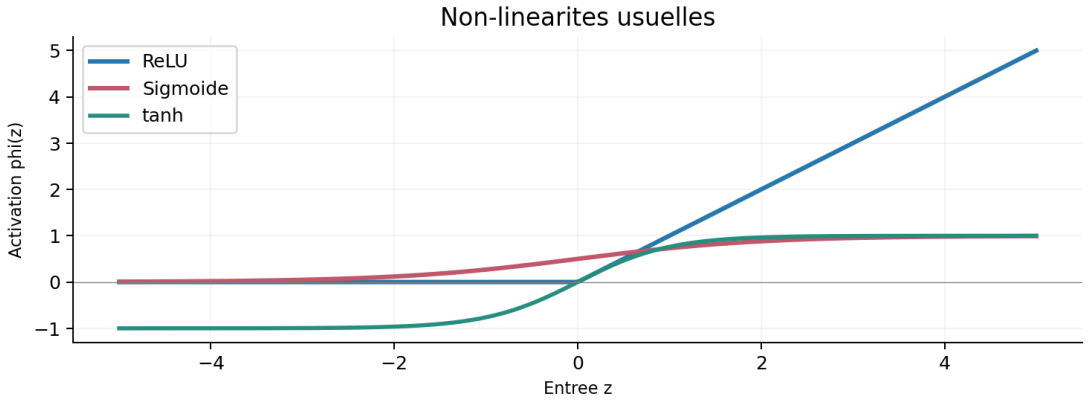


Figure 8: Fonctions d'activation usuelles : courbes mathématiques originales ; références [R03, R05].

10.2 Couches cachées et représentations

Une représentation cachée h n'est pas nécessairement une « notion humaine » identifiable. Elle constitue un ensemble de caractéristiques construites pour faciliter la tâche. Les neurones d'une couche peuvent réagir à des motifs statistiques utiles sans correspondre à des symboles interprétables individuellement. Dire qu'un réseau « comprend » une image ou une phrase est une métaphore utile, non une explication scientifique exhaustive.

10.3 Exemple de dimensions

Pour un MLP $4 \to 8 \to 3$: première couche $8 \times 4 + 8 = 40$ paramètres, seconde couche $3 \times 8 + 3 = 27$; soit **67 paramètres**. La sortie donne trois scores, que l'on transforme éventuellement

par softmax pour une classification. Le code doit toujours pouvoir afficher les dimensions de chaque activation : c'est l'un des diagnostics les plus efficaces en apprentissage profond.

Exercice. Calcule le nombre de paramètres d'un réseau $10 \to 16 \to 16 \to 2$. Pourquoi la disparition des non-linéarités change-t-elle radicalement l'expressivité ?

11. Backpropagation et optimisation des réseaux

À lire d'abord : Dive into Deep Learning (R03), *Backpropagation* ; [R17], *Autograd* et *Optimization*. **Ensuite :** [R05], chapitres sur l'entraînement.

11.1 La rétropropagation est une application de la règle de chaîne

Un réseau réalise un **forward pass** : entrées → activations → prédiction → perte. L'algorithme *backpropagation* calcule ensuite les gradients de la perte par rapport aux paramètres en partant de la fin, en appliquant la règle de chaîne. Les paramètres sont mis à jour par un **optimiseur**. Backpropagation **calcule** des gradients ; la descente de gradient ou Adam **les utilise** pour modifier les poids : ce sont deux opérations distinctes.

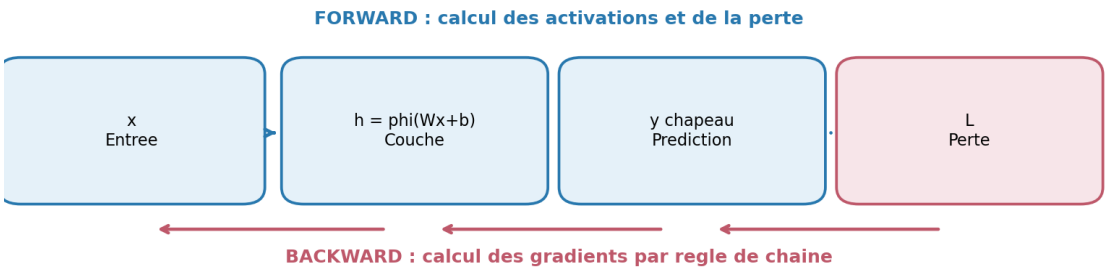


Figure 9: Chaîne de calcul et circulation des gradients : figure conceptuelle originale d'après [R03, R17].

11.2 Epoch, batch, SGD, Adam

Un **epoch** correspond à un passage sur les observations d'entraînement. Un **mini-batch** est un sous-ensemble traité pour estimer le gradient. Avec **SGD**, on réalise une mise à jour par mini-batch ; avec **Adam**, les gradients sont transformés au moyen d'estimations adaptatives de leurs moments. Adam demande moins de réglages dans de nombreux cas, mais n'est pas universellement supérieur à SGD.

Dans le code :

```
optimizer.zero_grad()        # effacer les anciens gradients
logits = model(batch_x)      # forward
loss = criterion(logits, batch_y)
loss.backward()              # dérivées via autograd
optimizer.step()             # mise à jour
```

Si tu oublies `zero_grad()` dans une boucle standard, les gradients peuvent **s'accumuler** d'une itération à l'autre ; cette accumulation est parfois volontaire dans des techniques avancées, mais souvent accidentelle chez un débutant.

11.3 Régularisation

Le **weight decay** pénalise les grands poids selon une variante de régularisation, le **dropout** masque aléatoirement certaines activations pendant l'entraînement, et l'**early stopping** arrête les itérations lorsque la performance de validation ne progresse plus suffisamment. Le *dropout* est généralement désactivé en phase d'évaluation : le comportement du modèle dépend donc de son mode (`train()` versus `eval()`).

Pièges : confondre nombre d'epochs et qualité, se fier uniquement à la loss d'entraînement, oublier le mode d'évaluation, ne pas fixer les seeds, comparer des réseaux entraînés avec des protocoles différents.

Exercice. Supposons un modèle très performant sur train mais médiocre sur validation. Donne trois hypothèses différentes et une expérience pour chacune.

12. PyTorch : tenseurs, autograd et boucle d'entraînement

À lire d'abord : PyTorch — Learn the Basics (R17), tutoriel officiel *Learn the Basics* dans l'ordre Tensors → Datasets → Model → Autograd → Optimization → Save/Load. **Ensuite :** [R03], notebooks PyTorch associés.

12.1 Représenter un calcul différentiable

Un `torch.Tensor` stocke valeurs, forme et type, ainsi qu'éventuellement un historique d'opérations permettant de différencier une quantité par rapport à ses paramètres. `requires_grad=True` demande le suivi automatique de certaines opérations. `loss.backward()` calcule les dérivées dans les attributs `.grad` des tenseurs feuilles pertinents. `nn.Module` permet d'encapsuler des couches et paramètres ; `DataLoader` itère sur des mini-batches.

12.2 Boucle minimale complète

```
import torch
from torch import nn

torch.manual_seed(0)
X = torch.randn(128, 2)
y = ((X[:, 0] + 0.6 * X[:, 1]) > 0).long()

model = nn.Sequential(nn.Linear(2, 8), nn.ReLU(), nn.Linear(8, 2))
criterion = nn.CrossEntropyLoss()  # prend des logits, pas une softmax préalable
optimizer = torch.optim.Adam(model.parameters(), lr=0.02)

for epoch in range(120):
    optimizer.zero_grad()
    logits = model(X)
    loss = criterion(logits, y)
    loss.backward()
    optimizer.step()
```

```
model.eval()
with torch.no_grad():
    accuracy = (model(X).argmax(dim=1) == y).float().mean().item()
print(f"Accuracy TRAIN : {accuracy:.3f}")
```

Cet exemple **mesure volontairement la précision sur les données d'entraînement** pour illustrer l'API ; ce n'est pas une évaluation valide de généralisation. Il faudra introduire un vrai split et comparer les performances hors échantillon.

12.3 Sauvegarde et reproductibilité

Sauvegarder un `state_dict` permet de persister les poids ; garder le code d'architecture, les versions des bibliothèques et les paramètres d'entraînement est indispensable pour restaurer correctement un modèle. Une seed aide à reproduire les tirages, mais **n'assure pas toujours une déterminisme intégral**, selon les opérations matérielles, systèmes et bibliothèques.

Exercice. Modifie le réseau en $2 \to 32 \to 2$; observe la loss. Puis fais volontairement une erreur : applique softmax avant `CrossEntropyLoss` et explique pourquoi le contrat de l'API ne correspond plus au calcul prévu.

Partie IV — Traitement du langage et modèles de langue

13. Du texte brut aux vecteurs : NLP classique

À lire d'abord : Speech and Language Processing — Jurafsky & Martin (R04), chapitres *Words and Tokens*, *N-gram Language Models* et *Text Classification* selon l'édition du manuscrit ; [R18], *Working with Text Data*. **Ensuite :** [R19], cours LLM Hugging Face, chapitres de base.

13.1 Un texte n'est pas une liste de concepts déjà compris

Un **corpus** est une collection de documents ; la **tokenisation** découpe le texte en unités (mots, sous-mots ou symboles, selon le système). Un **vocabulaire** est la liste d'unités reconnues. **Normaliser** signifie modifier la surface textuelle pour simplifier certaines variations, avec des risques : « pas bon » devient trompeur si l'on retire « pas ». Le *stemming* réduit des formes vers une racine approximative ; la *lemmatisation* vise une forme canonique selon une analyse linguistique. Le traitement ne doit pas détruire les indices utiles à la tâche.

13.2 Bag of Words et TF–IDF

Le **Bag of Words** représente chaque document par des comptes de mots : il perd largement l'ordre des mots, donc la syntaxe. Le **TF–IDF** combine la fréquence d'un terme dans un document et sa rareté au sein du corpus, sous diverses conventions numériques. Une forme pédagogique est $\text{tfidf}(t, d) = \text{tf}(t, d) \log\left(N / \text{df}(t)\right)$. Les implémentations utilisent souvent du lissage et de la normalisation : les formules exactes doivent être lues dans la documentation de l'outil.

Exemple : « film incroyable » et « film décevant » partagent une dimension `film`, mais leurs dimensions `incroyable`/`décevant` diffèrent. Les n-grams peuvent représenter « pas bon » comme une unité combinée ; ils ne résolvent pas pour autant l'ironie ou les dépendances longues.

13.3 Classifier un avis

Pipeline de base : nettoyage contrôlé → TF–IDF → régression logistique ou Linear SVM → classe prédite. Pour catégorie (film/livre/électronique) et sentiment (positif/neutre/négatif), on peut entraîner deux classificateurs distincts. Le **sentiment** n'est pas identique à une note en étoiles : l'étiquetage par notes est une approximation à expliciter.

```
from sklearn.pipeline import make_pipeline
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
pipeline = make_pipeline(
    TfidfVectorizer(ngram_range=(1, 2), min_df=1),
    LogisticRegression(max_iter=300)
)
pipeline.fit(["excellent film", "film ennuyant", "roman passionnant", "livre médiocre"],
```

```
            ["positif", "negatif", "positif", "negatif"])
print(pipeline.predict(["film passionnant"]))  # jouet : valeur non probante
```

Ce mini-exemple ne prouve **rien** sur la qualité d'un vrai classificateur : quatre phrases suffisent seulement à tester la plomberie de l'outil.

Exercice. Les phrases « J'aime ce film » et « Je n'aime pas ce film » se ressemblent fortement en Bag of Words. Donne deux améliorations possibles et explique leurs limites.

14. Embeddings, similarité et séquences

À lire d'abord : Speech and Language Processing — Jurafsky & Martin (R04), parties sur les représentations lexicales ; [R19], notions de tokenisation et de modèles. **Ensuite :** [R03], *Attention Mechanisms and Transformers*.

Un **embedding** associe à un mot, un sous-mot ou un document un vecteur dans $\mathbb{R}^d$. Contrairement à une représentation « one-hot », les vecteurs denses peuvent apprendre des proximités statistiques pertinentes. La **similarité cosinus** est $\cos(u, v) = \frac{u \cdot v}{\|u\|\|v\|}$. Une grande valeur indique un angle faible, pas nécessairement une équivalence sémantique parfaite.

Dans un modèle de langue, une **séquence** conserve l'ordre des tokens ; les embeddings de tokens sont souvent combinés à des informations de **position**. Un embedding **statique** attribue souvent une représentation identique à un token quelle que soit la phrase ; une représentation **contextualisée** dépend des autres tokens. Les modèles Transformers construisent ce second type de représentation par leurs couches d'attention.

Exemple calculé. Pour $u = (1, 0)$ et $v = (0, 1)$, la similarité cosinus vaut 0 ; pour $u = (1, 1)$ et $v = (2, 2)$ elle vaut 1. Cette propriété est géométrique. Associer proximité géométrique à proximité sémantique suppose que l'entraînement ait structuré l'espace de façon pertinente.

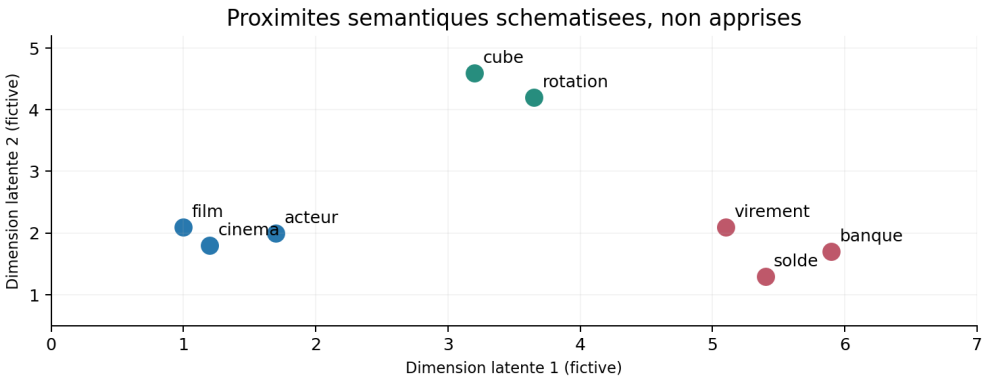


Figure 10: Projection schématique de vecteurs de mots artificiels : aucun embedding pré-entraîné n'a été utilisé. Références [R04, R19].

Exercice. Pourquoi ne faut-il pas comparer directement des embeddings générés par deux modèles non alignés ? Quelle information un modèle uniquement Bag of Words ne peut-il pas représenter sans transformation ?

15. Attention : comprendre précisément Q, K, V

À lire d'abord : Vaswani et al. — Attention Is All You Need (R20), Vaswani et al., *Attention Is All You Need* (2017), sections 3.1–3.2, puis [R04], chapitre Transformers. **Ensuite :** [R08], cours Stanford CS336 après maîtrise de PyTorch.

15.1 Intuition contrôlée

Lorsqu'un token est représenté dans une phrase, il peut devoir tenir compte d'autres tokens. Le mécanisme d'attention calcule **combien** chaque position doit contribuer à la représentation de chaque autre position. Les matrices Q (*queries*), K (*keys*) et V (*values*) sont trois transformations apprises des représentations d'entrée, **pas trois dictionnaires de mots indépendants**.

Pour une matrice de représentations $X \in \mathbb{R}^{T \times d_{model}}$, on construit $Q = XW_Q$, $K = XW_K$, $V = XW_V$. L'attention **scaled dot-product** s'écrit :

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}} + M\right)V.$$

Ici $QK^\top$ contient des scores de compatibilité entre positions ; la softmax transforme chaque ligne en poids positifs sommant à 1 ; la multiplication par V produit des combinaisons pondérées de valeurs. Le facteur $1/\sqrt{d_k}$ contrôle l'échelle typique des scores. M représente éventuellement un **masque**, par exemple un masque causal interdisant de regarder des tokens futurs. Lorsque le masque exclut une position, sa valeur est assimilée à $-\infty$ avant softmax.

15.2 Petit calcul d'attention

Supposons une query $q = (1, 0)$, deux keys $k_1 = (1, 0)$, $k_2 = (0, 1)$ et des values scalaires $v_1 = 10, v_2 = 20$. Les scores non normalisés sont $(1, 0)/\sqrt{2} \approx (0{,}707, 0)$. La softmax donne environ $(0{,}67, 0{,}33)$. La valeur agrégée est proche de $0{,}67 \times 10 + 0{,}33 \times 20 = 13{,}3$. L'attention **ne choisit pas simplement un seul token** : elle construit une moyenne pondérée de vecteurs de valeurs.

15.3 Self-attention, multi-head et coûts

La **self-attention** utilise Q, K, V issus de la même séquence (pas nécessairement des matrices de paramètres identiques). La **multi-head attention** apprend plusieurs projections en parallèle, pouvant extraire des dépendances distinctes. Pour une self-attention dense standard, la matrice des scores a une taille $T \times T$, d'où un coût mémoire et calcul qui croît quadratiquement avec la longueur de la séquence pour cette composante. Les architectures modernes possèdent de nombreuses optimisations : ne confonds pas cette caractéristique du mécanisme de base avec une règle absolue de toutes les implémentations actuelles.

Exercice. Si la séquence passe de 100 à 200 tokens, que devient le nombre de coefficients dans une matrice d'attention dense ? Pourquoi le masque causal est-il nécessaire lors de l'entraînement d'un modèle autoregressif ?

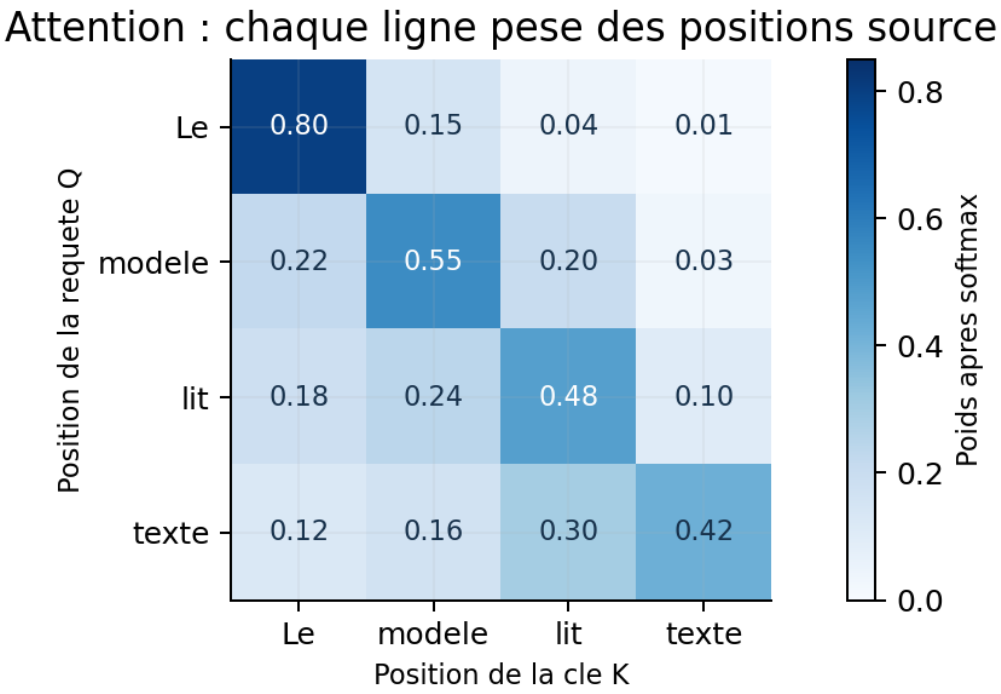


Figure 11: Matrice de poids d'attention illustrative à quatre tokens ; données fabriquées, figure originale fondée sur [R20].

16. Architectures Transformers, pré-entraînement et LLM

À lire d'abord : Speech and Language Processing — Jurafsky & Martin (R04), parties introduction aux LLM / Transformers / pré-entraînement ; [R19], premières unités du cours Hugging Face. **Ensuite :** [R20], article original ; [R08], Stanford CS336 (avancé).

Un **Transformer** superpose des blocs de transformation : mécanismes d'attention, réseaux feed-forward, résidus et normalisation, plus des embeddings de tokens et de positions. Dans un **encoder-only** (famille BERT), on apprend des représentations contextualisées destinées à diverses tâches. Dans un **decoder-only** autoregressif (famille GPT), on prédit le prochain token conditionnellement à un contexte précédent. Les systèmes *encoder-decoder* sont fréquents pour traduire ou transformer des séquences.

16.1 Prédire le prochain token

Pour des tokens $t_1, \dots, t_T$, l'objectif autoregressif maximise $\prod_{i=1}^{T} P(t_i \mid t_1, \dots, t_{i-1})$, ou minimise la **cross-entropy** moyenne sur les positions prédites. Le modèle génère des **logits** sur son vocabulaire, convertis en probabilités par softmax ; une procédure de **décodage** choisit le token suivant puis recommence.

La *temperature* transforme les logits z en z/τ avant softmax : une température faible rend généralement la distribution plus concentrée ; une température plus grande la rend plus plate. Le *top-k sampling* conserve un nombre fixe de tokens candidats ; le *top-p (nucleus) sampling* conserve un ensemble de tokens cumulant une masse de probabilité déterminée.

16.2 Pré-entraînement et adaptation

Le **pré-entraînement** construit un modèle général à partir de grands corpus. Le **fine-tuning** ajuste le comportement sur des données ou tâches spécifiques. L'**instruction tuning** utilise des exemples d'instructions et réponses ; l'alignement peut mobiliser d'autres méthodes d'apprentissage des préférences. RAG (*retrieval-augmented generation*) ajoute un système de recherche documentaire pendant la génération ; ce n'est **pas** la même chose qu'un fine-tuning, et il ne supprime pas mécaniquement les hallucinations.

16.3 Pourquoi un LLM n'est pas une base de connaissances certaine

Un LLM optimise une tâche statistique sur des données d'entraînement ; ses productions ne constituent ni des démonstrations formelles ni des observations vérifiées. Les erreurs peuvent provenir de l'ambiguïté du contexte, des données manquantes, du décodage, de connaissances obsolètes ou d'une demande non adaptée. Pour une application scientifique, l'**évaluation factuelle**, la traçabilité des sources et la capacité à s'abstenir sont des exigences d'ingénierie à part entière.

Exercice. Décris la différence entre *tokenizer*, *embedding*, *attention*, *logit*, *sampling* et *fine-tuning*. Pour quel type de besoin RAG est-il a priori plus pertinent que réentraîner un modèle ?

17. Construire et évaluer un mini modèle de langue

À lire d'abord : Speech and Language Processing — Jurafsky & Martin (R04), *N-gram Language Models* et génération ; [R03], chapitres Transformer. **Ensuite :** [R08] (niveau avancé).

Avant tout Transformer, construis un **modèle n-gram**. Une chaîne de caractères devient une série de symboles ; un modèle bigram estime $P(t_{i+1} \mid t_i)$ par comptage, éventuellement avec un lissage pour éviter les probabilités nulles. Une distribution est ensuite échantillonnée pour générer des suites. Cette expérience montre le lien entre **distribution conditionnelle**, **prédiction du prochain symbole** et **génération**, mais aussi la limite d'un contexte extrêmement court.

L'**entropie croisée** sur un jeu de test mesure la qualité de la distribution attribuée aux véritables tokens. La **perplexité**, $\exp(H)$ pour une entropie en logarithmes naturels, résume cette perte comme un nombre effectif moyen de possibilités ; elle dépend de la tokenisation et des données, donc comparer des perplexités de modèles à vocabulaires différents peut être trompeur. Une bonne perplexité ne garantit pas que le modèle soit factuellement correct ou sûr.

Mini expérience : compare un unigram, un bigram et un trigram sur un très court corpus, puis explique pourquoi une amélioration sur train ne suffit pas. Pour un mini-Transformer pédagogique, reproduis les étapes fondamentales (embedding, attention causale, projection de sortie, cross-entropy), mais reste sur des données et une architecture réduites.

Pièges courants. Fuite entre train et test à cause de segments de textes répétés, contamination d'évaluation, génération non reproductible, corpus sans licence claire, comparaison de modèles sur différents prompts.

Exercice. Si un modèle attribue les probabilités 0,8 puis 0,2 aux deux tokens corrects d'un exemple, calcule leur moyenne de $-\log p$ en logarithme naturel et explique le sens d'une forte pénalité sur le second token.

Partie V — Recherche d'états, décision et Rubik's Cube

18. Graphe d'états, permutations et problèmes de recherche

À lire d'abord : Berkeley CS188 — Introduction to AI (R06), UC Berkeley CS188 *Uninformed Search* ; [R07], Russell et Norvig, chapitre sur la résolution de problèmes. **Ensuite :** [R21], documentation du solveur en deux phases de Kociemba.

Un **état** décrit une configuration pertinente du monde ; une **action** fait passer d'un état à un autre par une **fonction de transition** $T(s, a) = s'$. L'**état initial** est le point de départ, l'**état but** satisfait une condition $G(s) =$ vrai. Le **graphe d'états** contient les états comme sommets et les transitions admissibles comme arêtes ; un chemin de ce graphe est une séquence d'actions.

Un Rubik's Cube n'est pas six grilles indépendantes. Son état possède des contraintes structurelles : les mouvements permutent des pièces et conservent certains invariants. Générer des couleurs de manière indépendante peut créer des configurations impossibles physiquement. Pour un mélange **garanti valide**, applique à l'état résolu une suite de rotations légales, en conservant une graine pseudo-aléatoire.

18.1 Représenter les permutations

Une **permutation** réarrange des éléments de manière bijective. Elle possède un inverse. Une séquence $a_1, a_2, \dots, a_k$ compose plusieurs permutations ; exécuter une rotation puis son inverse doit restaurer l'état. Dans le code, ce sont des **invariants testables** : $R \circ R^{-1} = \text{id}$ et $R^4 = \text{id}$ pour un quart de tour de face 3×3.

18.2 Graphe d'états vs arbre de recherche

Le même état peut être accessible par plusieurs suites de mouvements. Un **arbre de recherche** développe des chemins possibles, éventuellement redondants ; un **graphe** reconnaît que plusieurs chemins peuvent pointer vers le même état. Mémoriser les états visités évite certaines répétitions, mais coûte de la mémoire. Le **branching factor** b représente un nombre moyen d'actions possibles par étape ; explorer jusqu'à profondeur d peut demander de l'ordre de b^d nœuds dans le pire cas, ce qui explique l'explosion combinatoire.

Exercice. Avec 12 actions possibles par état et une profondeur de 5, calcule 12^5. Pourquoi le nombre d'états distincts explorés peut-il être plus faible que le nombre de chemins ?

19. BFS, DFS, IDDFS, A* et heuristiques

À lire d'abord : Berkeley CS188 — Introduction to AI (R06), cours *Uninformed Search* et *A Search and Heuristics** ; [R07], *Informed Search*. **Ensuite :** [R21], algorithme Kociemba.

BFS (*breadth-first search*) explore les profondeurs successives. Dans un graphe non pondéré, il trouve un plus court chemin en nombre d'arêtes si les conditions habituelles sont satisfaites, mais

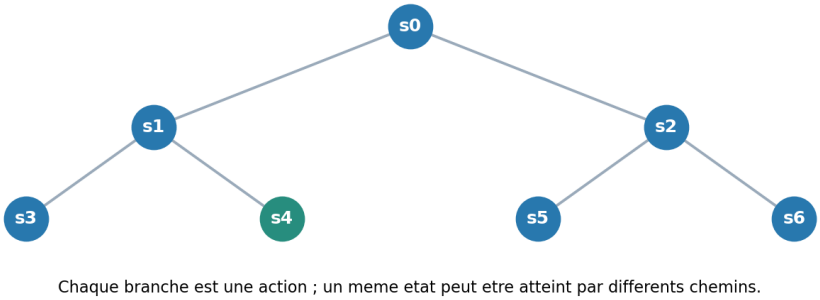


Figure 12: Arbre de recherche partiel : états illustratifs, sans prétention de représenter tout le cube. Figure originale ; cours Berkeley CS188 [R06].

consomme beaucoup de mémoire. **DFS** explore une branche avant de revenir ; il peut parcourir peu de mémoire par profondeur mais ne garantit pas un chemin minimal et peut ne pas terminer sur certains espaces infinis. **IDDFS** recommence une DFS avec des limites de profondeur croissantes : il échange des répétitions de calcul contre une meilleure empreinte mémoire.

19.1 La fonction d'évaluation de A*

A* classe les nœuds selon $f(n) = g(n) + h(n)$: g est le coût déjà parcouru, h une estimation du coût restant. Une heuristique **admissible** ne surestime jamais le coût optimal restant ; dans les conditions classiques d'A* et avec gestion appropriée des doublons, cela aide à garantir l'optimalité. Une heuristique trop faible explore beaucoup d'états ; une heuristique trop optimiste au mauvais sens (qui surestime) peut compromettre l'optimalité.

19.2 IDA* et réduction de l'espace

IDA* (iterative deepening A*) effectue des recherches successives avec des limites sur f plutôt que sur la profondeur seule. Les **pruning rules** éliminent des branches qui ne peuvent pas être utiles, par exemple des rotations inverses immédiatement consécutives. Une **pattern database** pré-calcule des coûts exacts pour une projection simplifiée de l'état, utilisée comme heuristique. La méthode de **Kociemba** réduit le cube en deux phases via des groupes d'états particuliers ; elle n'assure pas systématiquement un nombre minimal absolu de mouvements.

Différence scientifique essentielle : résoudre un cube avec une heuristique construite par raisonnement n'implique aucun entraînement. Si tu apprends une heuristique à partir de données, tu ajoutes alors un composant de ML à l'algorithme de recherche.

Exercice. Sur un graphe à poids uniformes, compare BFS et DFS si une solution courte existe mais qu'une branche très longue précède cette solution. Quel algorithme privilégierais-tu si la mémoire est très contrainte ?

20. Apprentissage par renforcement : extension, pas raccourci

À lire d'abord : Sutton & Barto — Reinforcement Learning (R22), Sutton et Barto, chapitres d'introduction à l'apprentissage par renforcement ; [R06], cours Berkeley sur MDP et fonctions de valeur. **Ensuite :** [R23], *Deep RL Course* de Hugging Face.

Un **processus décisionnel de Markov** (MDP) formalise états S, actions A, transitions $P(s' \mid s, a)$, récompenses r et facteur d'actualisation $\gamma \in [0, 1]$. La **politique** $\pi(a \mid s)$ spécifie comment l'agent choisit ses actions. L'objectif est d'optimiser le retour attendu $G_t = \sum_{k \geq 0} \gamma^k r_{t+k+1}$. La fonction **valeur** $V^\pi(s)$ estime le retour depuis un état sous une politique ; $Q^\pi(s, a)$ ajoute l'action initiale.

La **relation de Bellman** exprime une valeur en fonction d'une récompense immédiate et de valeurs futures. Dans le **Q-learning**, une mise à jour tabulaire classique est :

$$Q(s, a) \leftarrow Q(s, a) + \alpha \left[r + \gamma \max_{a'} Q(s', a') - Q(s, a) \right].$$

L'agent doit arbitrer entre **exploration** (tester des actions peu connues) et **exploitation** (choisir la meilleure action estimée). Ce cadre pourrait théoriquement servir à chercher des stratégies de cube, mais un 3×3 complet présente un espace d'états gigantesque. Pour un premier projet RL, commence par une grille ou un cube 2×2 simplifié avec une profondeur de mélange limitée.

Exercice. Avec $Q = 2$, récompense $r = 1$, $\gamma = 0{,}9$, $\max Q(s', \cdot) = 4$ et $\alpha = 0{,}1$, effectue une mise à jour Q-learning. Pourquoi un mauvais design de la récompense peut-il apprendre un comportement indésirable ?

Partie VI — Fiabilité, ingénierie et grands défis

21. Ingénierie expérimentale et reproductibilité

À lire d'abord : scikit-learn — Common Pitfalls (R15), scikit-learn *Common Pitfalls* ; [R17], PyTorch *Save/Load*. **Ensuite :** [R24], NIST AI RMF (cadre de gestion des risques).

Un prototype d'IA sérieux ne se réduit pas à `model.fit()`. Il doit préciser : la question scientifique, le protocole, la provenance des données, les unités, les hypothèses, les métriques, le matériel, les versions logicielles, les seeds et la méthode de comparaison. Une **baseline** donne un point de départ. Une **ablation study** supprime ou modifie un composant afin d'estimer son rôle ; elle doit conserver autant que possible les autres facteurs constants.

21.1 La fiche d'expérience

Rédige avant d'exécuter : « Hypothèse H1 : ajouter une variable d'historique améliore le rappel à précision fixée ». Ensuite, renseigne : jeu de données, split, variables disponibles en situation réelle, modèle, hyperparamètres, métriques, résultat, intervalle de variabilité ou répétitions, conclusion et suite. Cette habitude réduit les expériences opportunistes où l'on change simultanément données, paramètres et méthode d'évaluation.

21.2 Traçabilité du code

Utilise Git pour les changements de code, un environnement Python documenté, des tests unitaires pour les transformations de données, des tests d'intégration pour le pipeline, et des **tests de données** (colonnes requises, types, valeurs impossibles). Un README devrait permettre à un autre étudiant de réexécuter l'expérience. Lorsqu'un dataset possède une licence, respecte ses restrictions ; ne pousse pas de données confidentielles sur GitHub. Le notebook est pratique pour explorer, mais le code réutilisable gagne à être déplacé vers un module Python testé.

21.3 Mesurer l'incertitude des résultats

Un score observé est une réalisation dépendant du dataset, du tirage aléatoire et du protocole. Répéter certaines expériences avec différentes seeds et fournir des intervalles est plus informatif que d'annoncer trois décimales sur un petit test. Attention : les données temporelles, les groupes et les dépendances rendent certaines procédures de bootstrap ou de CV invalides si elles sont appliquées sans réflexion.

Exercice. Ton nouveau modèle gagne deux points de F1 sur un seul split aléatoire. Quels contrôles exigerais-tu avant d'écrire « amélioration confirmée » ?

22. Données réelles, déséquilibre, temps et changement de distribution

À lire d'abord : An Introduction to Statistical Learning — Python (R01), validation et évaluation ; [R15], problèmes de fuite ; [R24], management des risques. **Ensuite :** documentation spécialisée de ton domaine (énergie, banque, satellite).

Dans le monde industriel, la difficulté principale n'est pas toujours l'architecture du modèle. Les labels peuvent être retardés, bruités ou incomplets ; les populations changent ; les capteurs tombent en panne ; les coûts d'erreur varient. On distingue couramment un **covariate shift** (distribution des entrées $P(X)$ modifiée), un **label shift** ($P(Y)$ modifié dans certaines hypothèses) et un **concept drift** (relation entre entrées et sortie qui évolue). Ces étiquettes décrivent des cas théoriques distincts, pas toujours observables directement.

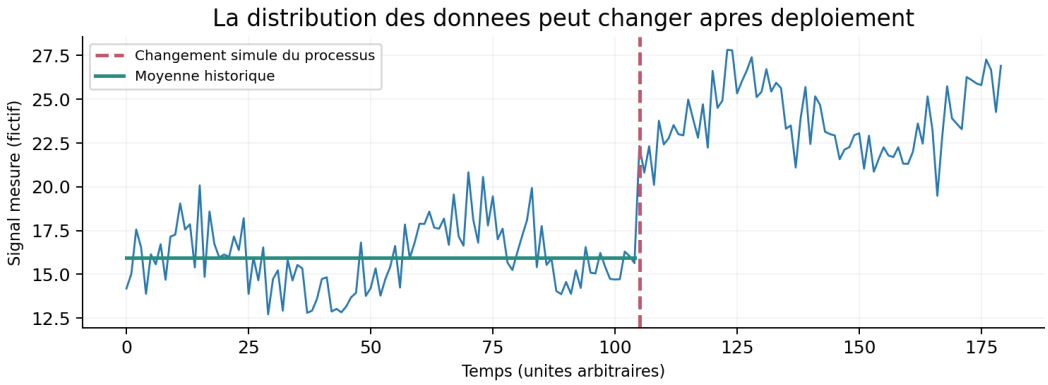


Figure 13: Une relation prédictive qui change au fil du temps : série entièrement simulée, illustrant un concept drift. Références [R01, R24].

Un système d'alerte doit aussi gérer le **déséquilibre des classes** : une très petite proportion de fraudes, pannes ou incidents peut rendre l'accuracy peu informative. La présence d'un dataset synthétique est une aide pédagogique, pas une validation dans un contexte bancaire réel. Les métriques doivent être reliées à des conséquences : perte financière, délai de réponse, précision de localisation, énergie consommée, etc.

Exercice. Établis trois différences entre un benchmark offline et un service utilisé en continu. Que dois-tu contrôler si la proportion de fraude double six mois après l'entraînement ?

23. RAG, évaluation des assistants et cybersécurité des systèmes d'IA

À lire d'abord : Hugging Face — LLM Course (R19), cours Hugging Face pour modèles ; [R24], NIST AI RMF ; [R25], OWASP Top 10 for LLM Applications. **Ensuite :** [R08], Stanford CS336 (pour les fondements du modèle lui-même).

Un système **RAG** suit en général : collecte documentaire → découpage (*chunking*) → représentation/recherche → sélection de passages → génération avec contexte. Son évaluation sépare deux problèmes : **retrieval quality** (les bons documents sont-ils retrouvés ?) et **answer quality** (la réponse s'appuie-t-elle correctement sur ces documents ?). Mesures utiles : Recall@k sur documents attendus, précision des citations, taux d'abstention appropriée, taux de réponses non justifiées et temps de réponse.

Le **prompt injection** correspond à des instructions malveillantes placées dans du contenu non fiable, par exemple un document récupéré ou une page Web, pour détourner le comportement du système. Les documents doivent être traités comme **données**, jamais comme des directives de confiance équivalentes à celles de l'utilisateur et de l'application. Des politiques d'accès aux sources, la séparation des privilèges, les validations des actions externes et les tests adversariaux sont nécessaires.

Projet ambitieux mais mesurable : un assistant scientifique qui répond seulement à partir d'un corpus autorisé, cite les passages exacts, indique son incertitude et s'abstient si aucune preuve perti-

nente n'est présente. Un démonstrateur convaincant pourrait tester 100 questions réparties entre questions répondables, ambiguës et sans réponse. Il faudrait auditer les citations à la main sur un échantillon.

Exercice. Pourquoi une réponse avec cinq citations peut-elle malgré tout être fausse ? Conçois un protocole pour mesurer la proportion de citations réellement justificatives.

24. Robotique, énergie et vision : quelles théories avant un grand défi ?

À lire d'abord : spécialité choisie, documentation officielle et cours ciblés ; **ne cherche pas** à tout assimiler avant de produire une première simulation. Pour la robotique : Berkeley CS188 — Introduction to AI (R06) pour planification, [R22] pour RL ; pour la vision : [R26], Stanford CS231n ; pour l'industrialisation : [R24].

24.1 Robots et contrôle

Un robot ajoute au ML une **dynamique physique** : l'état peut inclure position, vitesse, attitude et énergie disponible. Un **contrôleur** transforme un écart de consigne en actions, avec stabilité et contraintes. Un véhicule à décollage vertical hybride ne se résume pas à une IA qui « pilote » ; il exige la modélisation des forces, la gestion des transitions, des capteurs, des actionneurs et de nombreuses questions de sûreté. Une **simulation** constitue une première étape sûre et vérifiable ; elle ne prouve pas une aptitude au vol réel.

24.2 Énergie et prévision

Pour un micro-réseau, prédire une demande d_t est distinct d'**optimiser** une décision de production et de stockage u_t sous contraintes. L'optimisation peut chercher à minimiser le coût $\sum_t c_t(u_t)$ tout en respectant des contraintes comme l'état de charge d'une batterie, la puissance maximale ou la continuité de service. La performance n'est pas seulement une erreur prédictive : elle inclut coût, résilience et violations de contraintes.

24.3 Satellites et vision

Une image satellite combine données spatiales, canaux spectraux et métadonnées géographiques. Un modèle de segmentation produit une catégorie pour chaque pixel ; un **IoU** (*intersection over union*) vaut $|A \cap B|/|A \cup B|$ pour comparer une région prédite A à une vérité terrain B. Attention aux différences d'illumination, de saison, de résolution, de couverture nuageuse et de localisation entre train et test. Une belle carte de prédiction n'est pas une validation scientifique.

Exercice. Pour chacun des trois domaines, propose une métrique de réussite, une contrainte de sécurité et une limite qu'un prototype purement logiciel ne permettra pas de lever.

25. Risques, limites et responsabilité de l'ingénieur IA

À lire d'abord : NIST — AI Risk Management Framework (R24), NIST AI RMF ; [R15], pièges expérimentaux ; [R25], sécurité des applications LLM. **Ensuite :** obligations légales et réglementaires applicables au pays, au contexte et au système concerné.

La **robustesse** décrit la sensibilité aux perturbations ; la **fiabilité** demande de tenir les performances dans le contexte attendu ; l'**équité** interroge les écarts de performance et de traitement selon des populations pertinentes ; la **confidentialité** concerne les données et informations potentiellement exposées ; l'**explicabilité** cherche des raisons interprétables des prédictions. Aucune de ces propriétés n'est automatiquement garantie par un score de précision élevé.

Le cadre NIST AI RMF 1.0 propose notamment les fonctions **Govern, Map, Measure, Manage**. Il s'agit d'un cadre de gestion des risques, pas d'une certification automatique. Pour un projet étudiant, retiens trois réflexes : documenter les populations et situations non couvertes, expliciter les coûts des erreurs, établir un processus de correction et de retrait du modèle si les résultats se détériorent.

Cas limites : un outil de détection de fraude peut bloquer des usagers légitimes ; un détecteur satellitaire peut ignorer certaines régions ; un assistant scientifique peut inventer une référence ; un robot peut échouer hors des conditions simulées. Le but n'est pas d'interdire la recherche, mais de rendre ses limites visibles et ses expériences honnêtes.

Exercice. Rédige une « model card » de dix lignes pour ton classificateur de fraude : tâche, données, population couverte, mesure, limites, risques et utilisation déconseillée.

26. Traduire la science en gestion de projet et leadership

À lire d'abord : The Scrum Guide (R27), *The Scrum Guide* pour la terminologie produit et cycles ; [R24], cadre de risques NIST pour la gouvernance technique. **Ensuite :** [R28], *The Manager's Path* (ouvrage payant, facultatif).

Un responsable technique ne doit pas seulement écrire du code. Il doit clarifier le **problème** avant la solution, identifier les parties prenantes, expliciter le **périmètre**, définir des résultats mesurables et communiquer les incertitudes. Un *backlog* liste les éléments de travail ; un *milestone* est un jalon vérifiable ; un *risk register* rassemble risque, probabilité, impact et mitigation ; une *decision record* explique pourquoi une architecture a été choisie et avec quelles alternatives.

26.1 La note de cadrage en six questions

1. Quel problème utilisateur ou scientifique veut-on résoudre ? 2. Quelle preuve montrerait que le prototype progresse ? 3. Quelles contraintes de temps, données, sécurité et coût s'imposent ? 4. Quels sont les principaux risques ? 5. Quel livrable démontrable peut-on produire dans un mois ? 6. Quel signal nous dirait qu'il faut arrêter ou changer d'approche ?

26.2 Rituels compatibles avec tes études

- **Début de semaine (10 min)** : un objectif, une hypothèse, un résultat attendu.
- **Après une expérience** : entrée de journal avec ce qui a marché et échoué.
- **Fin de semaine (20 min)** : démonstration du programme, métriques, blocages, prochaine priorité.
- **Fin de mois (45 min)** : *review* du résultat, *retrospective* sur ta manière de travailler, décision de poursuite ou bifurcation.

Les « soft skills » ne se valident pas en lisant une théorie sur le leadership : entraîner-toi à présenter un compromis à un camarade, à expliquer une limitation sans jargon et à faire relire ton protocole. Être chef de projet n'est pas équivalent à être le meilleur programmeur ; c'est en partie **rendre les décisions collectives claires et les risques maîtrisables**.

Exercice. Rédige une décision d'architecture : « TF–IDF + régression logistique ou Transformer pré-entraîné pour classifier des avis ? » Compare données, calcul, métriques, maintenance, licences et délai.

Partie VII — Exercices de synthèse et corrigés

Partie VI bis — Compléments conceptuels pour aller plus loin

Ces approfondissements ne constituent **pas** de nouvelles obligations pour le premier mois. Ils répondent à des questions qui apparaîtront en construisant les projets, et évitent que des concepts structurants restent au niveau de simples définitions.

Approfondissement A — Régularisation, convexité et stabilité numérique

Source prioritaire : Mathematics for Machine Learning (R02), chapitres *Vector Calculus* et *Continuous Optimization* ; [R01], *Linear Model Selection and Regularization*. **Source secondaire :** [R03], optimisation.

Pourquoi le minimum de la perte n'est pas le seul objectif

L'optimisation numérique tente de minimiser une fonction de coût ; l'apprentissage vise la **performance sur des observations nouvelles**. Ces deux objectifs ne coïncident pas toujours. Un modèle très souple peut réduire presque à zéro l'erreur d'entraînement, sans apprendre une relation qui se généralise. La **régularisation** ajoute une préférence pour certaines solutions : paramètres plus petits, modèle plus simple, poids moins nombreux ou moins instables.

La **ridge regression** ajoute une pénalité quadratique $\lambda\|w\|_2^2$ à la perte :

$$\min_{w,b} \frac{1}{n}\sum_i (y_i - w^\top x_i - b)^2 + \lambda\|w\|_2^2.$$

La **lasso** utilise plutôt $\lambda\|w\|_1 = \lambda\sum_j |w_j|$. Dans certains contextes, elle encourage des coefficients exactement nuls, ce qui peut simplifier l'interprétation. Il ne s'agit pas d'une preuve automatique qu'une variable est causalement inutile : la sélection dépend de corrélations, de pénalisation et de données.

Convexité et Hessienne

Une fonction f est **convexe** si $f(tx + (1-t)y) \leq tf(x) + (1-t)f(y)$ pour $t \in [0,1]$; visuellement, le segment joignant deux points du graphe reste au-dessus du graphe. Pour une fonction deux fois dérivable, une **Hessienne** positive semi-définie est un critère classique suffisant de convexité sur un domaine convexe. Une régression linéaire entraînée par MSE correspond à un problème convexe en ses poids ; un réseau de neurones profond n'a généralement pas cette structure globale simple.

La **Hessienne**, matrice des dérivées secondes, décrit la courbure locale. Une courbure très différente selon les directions peut rendre l'optimisation sensible au learning rate. Une normalisation appropriée des entrées améliore souvent le conditionnement d'un problème ; elle n'est pas un simple détail de présentation.

Exemple. $L(w) = (w-3)^2 + \lambda w^2$. La dérivée est $2(w-3) + 2\lambda w$. Le minimum est $w^* =$

$3/(1+\lambda)$. Lorsque λ augmente, la valeur optimale se rapproche de zéro : la régularisation a introduit une préférence mesurable, au prix d'un certain biais.

Petit test. Résous le même problème pour $\lambda = 0, 1$ et 9. Explique pourquoi le plus petit coefficient n'est pas forcément celui qui minimise la MSE sur les données d'entraînement.

Approfondissement B — PCA, clustering et apprentissage non supervisé

Source prioritaire : An Introduction to Statistical Learning — Python (R01), chapitre *Unsupervised Learning*. **Source secondaire :** [R02], *Principal Component Analysis* ; [R14], scikit-learn.

Réduire la dimension

Lorsque chaque observation possède des dizaines ou centaines de variables, il peut être utile de construire un espace plus petit qui conserve une grande partie des variations. L'**analyse en composantes principales (PCA)** calcule des directions orthogonales maximisant successivement la variance des données projetées. Les données doivent généralement être **centrées**, et parfois mises à l'échelle lorsque les unités des variables sont différentes.

La première direction principale correspond au vecteur propre associé à la plus grande valeur propre de la matrice de covariance, dans la formulation classique. Si X est centré et que v_1 est cette direction, la première composante est $z_1 = Xv_1$. La PCA est une transformation **linéaire non supervisée** : elle ignore les étiquettes de fraude ou de sentiment. Une forte variance conservée ne signifie donc pas nécessairement que les informations utiles à une classification ont été conservées.

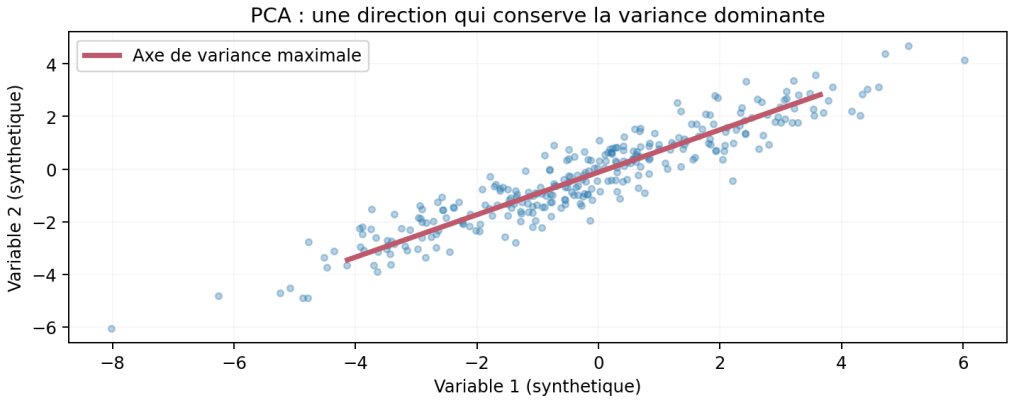


Figure 14: Une direction principale épouse la variance dominante des points : données synthétiques. Figure originale, méthode expliquée dans [R01, R02].

Regrouper des observations

Le **clustering** cherche des groupes selon une métrique de similarité. L'algorithme **k-means** alterne affectation des observations au centre le plus proche, puis recalcul des centres. Il minimise typiquement la somme des distances quadratiques aux centres ; son résultat dépend du nombre de clusters k, de l'initialisation et de la géométrie des données. Des clusters **ne sont pas** des catégories métier découvertes avec certitude.

Application prudente à la fraude. Un cluster d'opérations rares n'est pas automatiquement un cluster de fraudes. Une transaction légitime atypique peut être parfaitement normale. Pour une alerte, il faut un protocole de validation avec des cas labellisés ou une expertise métier.

Exercice. Deux colonnes représentent un montant en euros et une durée en secondes. Pourquoi faire une PCA ou un k-means sans tenir compte des échelles peut-il fausser la géométrie ? Quel diagnostic ferais-tu avant de choisir k ?

Approfondissement C — CNN, vision et segmentation

Source prioritaire : Dive into Deep Learning (R03), chapitres de réseaux convolutifs ; [R26], Stanford CS231n. **Source secondaire :** [R17], tutoriels modèles d'image PyTorch.

Un **réseau convolutif** (*CNN*) exploite la structure spatiale d'une image en appliquant des filtres locaux partagés à de nombreuses positions. Dans un exemple 2D simple, un noyau 3×3 parcourt l'image et produit, par produit local/somme, une **carte de caractéristiques**. Par convention, nombre d'implémentations de CNN calculent en réalité une **corrélation croisée** (sans retourner le noyau), souvent appelée « convolution » dans la documentation. Les **poids partagés** permettent de détecter un motif à plusieurs positions avec le même petit ensemble de coefficients.

Un **stride** supérieur à un saute certaines positions ; un **padding** ajoute des valeurs au bord pour contrôler la taille de sortie. La **dilatation** espace les coefficients d'un noyau, étendant son champ de vision sans multiplier proportionnellement le nombre de paramètres. Le **pooling** réduit parfois la résolution des activations ; ce n'est pas un composant universel de toutes les architectures modernes.

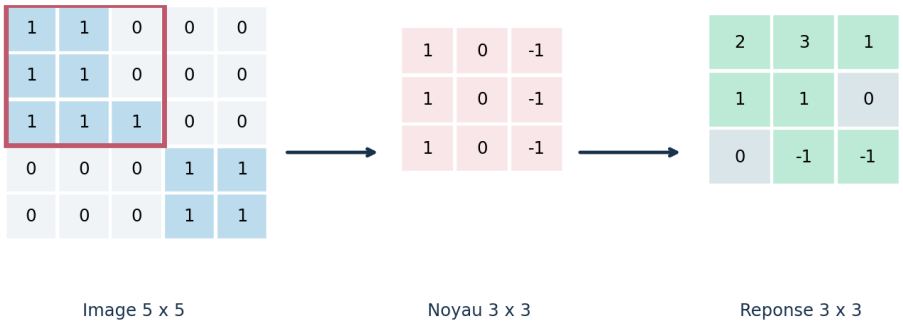


Figure 15: Un filtre 3x3 se déplace sur une grille 5x5 et produit une carte de réponses ; figure originale d'après les notions de convolution [R03, R26].

Pour la **classification**, le modèle rend un label global ; pour la **segmentation**, il étiquette les pixels ou régions. La **détection d'objets** renvoie souvent des boîtes et des scores. Ces tâches ont des métriques différentes : accuracy sur labels, IoU/Dice sur régions, mesures de précision/rappel adaptées aux objets. En imagerie satellitaire, une séparation aléatoire par pixel produirait souvent une validation trop optimiste à cause de la forte dépendance spatiale : il faut plutôt construire des splits par scènes, lieux ou périodes.

Exemple pratique. En Python, prends une image synthétique de 5×5 et applique un noyau de détection de bord vertical. Observe la réponse quand tu inverses le contraste ; puis vérifie qu'un filtre appris ne détecterait pas forcément la même notion hors des images de son domaine d'entraînement.

Approfondissement D — Séries temporelles et prévision de demande

Source prioritaire : An Introduction to Statistical Learning — Python (R01), chapitres sur prédiction et validation ; [R14], modèle sélection/cross-validation. **Source secondaire :** manuel spécialisé de

prévision temporelle du domaine choisi.

Une **série temporelle** est une suite d'observations ordonnées y_t où le temps et les dépendances passées importent. On distingue **tendance**, **saisonnalité**, fluctuations et événements exceptionnels. Des **lags** sont des observations passées utilisées comme entrées, par exemple y_{t-1} et y_{t-24} pour une consommation horaire. Une **fenêtre glissante** prend plusieurs observations passées pour prédire une ou plusieurs valeurs futures.

Les splits aléatoires usuels peuvent créer une fuite temporelle : on entraînerait sur des observations futures pour évaluer des prédictions du passé. Le **backtesting** ou la validation temporelle simulent une succession de moments où seules les informations passées sont disponibles. Une prévision *one-step ahead* (une période) n'est pas la même tâche qu'une prévision 24 heures à l'avance.

Une **baseline temporelle** utile est souvent la dernière observation (*naive forecast*) ou l'observation correspondant à la même heure du jour précédent (*seasonal naive*). Pour un réseau énergétique, battre une baseline naïve sur MAE ne suffit pas ; les erreurs pendant les pics peuvent être plus importantes que la moyenne.

Exercice. Tu disposes de consommations horaires $y_1, \dots, y_{200}$. Décris un test qui prédit chaque heure des indices 151 à 200 avec uniquement des informations disponibles avant l'heure prédite.

Approfondissement E — D'une représentation à une décision sous contraintes

Source prioritaire : Mathematics for Machine Learning (R02), optimisation ; [R06], recherche et MDP ; [R24], gestion des risques. **Source secondaire :** documentation de simulation du domaine (robotique ou énergie).

Dans les grands défis, un modèle d'IA est fréquemment une pièce d'une chaîne plus large : **observer**, **estimer**, **prévoir**, **décider**, **agir**, **surveiller**. Un système énergétique peut prédire une demande future, mais la décision finale de charge/décharge dépend aussi des contraintes physiques. Formellement, on peut écrire :

$$\min_{u_1,\dots,u_T} \sum_{t=1}^{T} c_t(u_t, \hat{d}_t) \quad \text{sous contraintes } g_t(u_t, s_t) \le 0.$$

Ici $\hat{d}_t$ est une prédiction, u_t une décision et s_t l'état du système. Une solution mathématiquement optimale **pour le modèle** peut être mauvaise si les prévisions sont fausses ou si des contraintes réelles n'ont pas été modélisées. La planification robuste considère explicitement des scénarios d'incertitude ou des marges de sécurité.

En robotique, la **commande en boucle fermée** mesure l'écart entre l'état souhaité et l'état observé, puis corrige l'action ; la boucle ouverte exécute une séquence sans tenir compte des perturbations mesurées. Un simulateur peut aider à étudier la stabilité et les marges, mais les performances simulées n'équivalent pas à des essais en matériel réel.

Exercice. Un modèle de consommation prédit 10 kWh avec une incertitude de ±3 kWh. Pourquoi programmer une batterie pour fournir exactement 10 kWh peut-il être fragile ? Propose une contrainte de sécurité mesurable.

Approfondissement F — Transformer minimal : trajet des dimensions

Source prioritaire : Vaswani et al. — Attention Is All You Need (R20), Vaswani et al. ; [R04], chapitre Transformers ; [R03], *Attention Mechanisms and Transformers*. **Niveau avancé ensuite :** [R08], Stanford CS336.

Considérons un batch de taille B, une longueur de séquence T, une dimension de représentation d_{model}. Après tokenisation, les identifiants ont souvent la forme (B, T). Une table d'embedding les transforme en tenseur (B, T, d_{model}). Pour une tête d'attention de dimension d_k, on calcule Q et K de forme (B, T, d_k), et V de forme (B, T, d_v). Les scores $QK^\top$ (transposition des deux dernières dimensions) donnent une matrice (B, T, T) ; le résultat après pondération des valeurs donne (B, T, d_v).

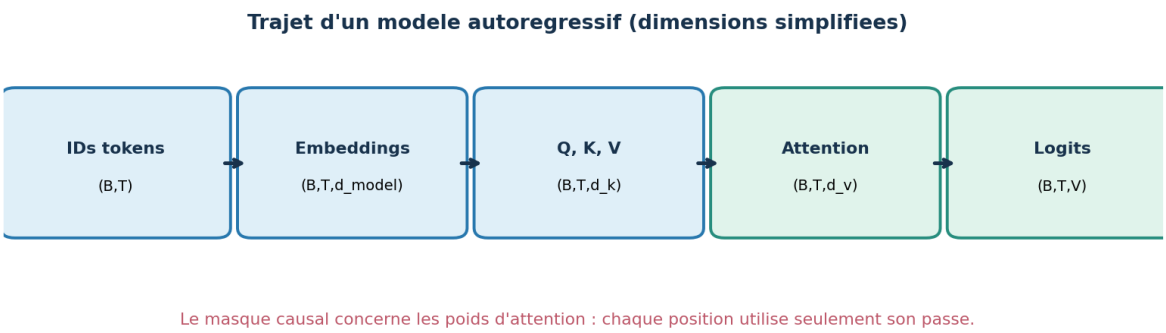


Figure 16: Trajet d'une séquence dans un bloc de Transformer : dimensions explicites, schéma original d'après [R20, R04].

En multi-head attention, plusieurs têtes produisent des sorties qui sont concaténées puis projetées vers d_{model}. Un **bloc résiduel** ajoute la sortie d'une sous-couche à une représentation compatible en dimension : $x \mapsto x + F(x)$. La **normalisation** stabilise certaines caractéristiques de l'entraînement ; sa position exacte (pré-normalisation ou post-normalisation) dépend de l'architecture. Chaque token traverse ensuite un réseau feed-forward (*MLP*) appliqué position par position, qui peut mélanger les canaux de représentation sans mélanger directement les positions.

Pour un **decoder-only causal**, un masque interdit à une position i d'utiliser directement les positions futures $j > i$. À l'entraînement, toutes les positions d'une séquence peuvent être traitées simultanément dans un passage parallèle sous ce masque ; en génération autoregressive, chaque nouveau token dépend des tokens précédemment choisis.

L'étape finale projette les représentations dans une dimension égale à la taille du vocabulaire V, donnant (B, T, V) logits. La cross-entropy compare les logits à la séquence **décalée d'une position** pour apprendre la prédiction du prochain token.

Exercice. Pour $B = 8, T = 128, d_{model} = 256, d_k = 64$ et $V = 16\,000$, donne la forme de Q, de la matrice des scores et des logits de sortie. Explique le rôle du masque et pourquoi l'attention ne suffit pas, seule, à tout définir de l'architecture.

27. Exercices corrigés, de la définition au calcul

Les exercices ci-dessous ne dispensent pas de coder : ils contrôlent surtout la cohérence des raisonnements. Pour chaque question, essaie de répondre **avant** de lire la correction, puis mets en œuvre un test Python.

Exercice A — Dimensions d'une couche

Question. Un lot contient 64 observations, chacune avec 12 variables. Une couche linéaire produit 7 sorties. Quelle est la forme des poids et combien de paramètres possède-t-elle ?

Correction. Avec la convention PyTorch `Linear(in_features=12, out_features=7)`, la matrice des poids est de forme $(7, 12)$; il y a 84 poids et 7 biais, soit **91 paramètres**. La sortie sur le lot a la forme $(64, 7)$. Un écart entre ces dimensions indique une erreur dans le modèle ou la représentation des données.

Exercice B — Une mise à jour de descente de gradient

Question. $L(w) = (w-5)^2$, $w_0 = 1$, $\eta = 0{,}2$. Calcule w_1 et les deux pertes.

Correction. $L'(w) = 2(w-5)$, donc $L'(1) = -8$; $w_1 = 1 - 0{,}2(-8) = 2{,}6$. $L(w_0) = 16$ et $L(w_1) = 5{,}76$. Attention, un pas plus grand ne garantit pas toujours une descente ; teste $\eta = 1{,}2$.

Exercice C — Probabilités conditionnelles et fraude rare

Question. Une fraude apparaît dans 0,1 % des opérations, la sensibilité est 90 %, le taux de faux positifs 5 %. Quelle est la précision approximative ?

Correction. Sur 100 000 transactions, il y a 100 fraudes dont 90 détectées et 99 900 transactions légitimes dont 4 995 faussement signalées. Précision $= 90/(90 + 4\,995) \approx 1{,}77\%$. Le changement de prévalence transforme radicalement l'intérêt opérationnel du détecteur sans changer ses propriétés conditionnelles supposées.

Exercice D — Classification et F1

Question. TP=18, FP=6, FN=12, TN=164. Calcule précision, rappel et F1.

Correction. Précision $18/24 = 0{,}75$, rappel $18/30 = 0{,}6$, F1 $= 2(0{,}75)(0{,}6)/(1{,}35) = 2/3 \approx 0{,}667$. L'accuracy vaut $182/200 = 0{,}91$ et cache une capacité limitée à détecter la classe positive.

Exercice E — Dérivation en chaîne

Question. $L(w) = (2w-3)^2$. Calcule la dérivée et sa valeur en $w = 1$.

Correction. Avec $u(w) = 2w - 3$, $L' = 2u\,u' = 4(2w-3)$. Donc $L'(1) = -4$. Vérification numérique : pour h petit, $[L(1+h) - L(1-h)]/(2h) \approx -4$.

Exercice F — Normalisation et fuite

Question. Tu veux normaliser une variable avec moyenne μ et écart-type σ. Tu calcules μ, σ sur l'ensemble train + test. Pourquoi est-ce problématique ?

Correction. Tu as utilisé une information statistique provenant du test pour construire la transformation, donc ton protocole ne reproduit plus une prédiction sur données réellement nouvelles. Ajuste μ, σ uniquement sur train, puis applique cette transformation inchangée à validation/test. Utilise un `Pipeline` si possible.

Exercice G — Softmax sur deux scores

Question. Pour les logits $(0, \ln 3)$, calcule les probabilités softmax.

Correction. $e^0 = 1$, $e^{\ln 3} = 3$, donc $p = (1/4, 3/4) = (0{,}25, 0{,}75)$. La softmax renormalise les exponentielles, elle n'« ajoute » pas de probabilités prédéfinies aux logits.

Exercice H — Nombre de paramètres d'un réseau

Question. Réseau dense $10 \to 16 \to 16 \to 2$, avec biais par couche.

Correction. Première couche : $10 \times 16 + 16 = 176$; deuxième : $16 \times 16 + 16 = 272$; sortie : $16 \times 2 + 2 = 34$. Total : **482 paramètres**.

Exercice I — Matrice d'attention

Question. Combien de coefficients une tête d'attention dense calcule-t-elle pour une séquence de 128 tokens ? Et de 256 ?

Correction. $128^2 = 16\,384$, puis $256^2 = 65\,536$ coefficients. Le doublement de longueur multiplie cette partie par **4**. Les masques et optimisations modifient la manière de calculer ou stocker l'opération, pas les dépendances conceptuelles de la formule dense.

Exercice J — Perplexité

Question. Un modèle attribue 0,8 puis 0,2 aux deux tokens cibles. Quelle est la perte moyenne en log naturel ?

Correction. $-\frac{1}{2}(\ln 0{,}8 + \ln 0{,}2) \approx 0{,}9163$. Perplexité : $e^{0{,}9163} \approx 2{,}5$. Ce chiffre dépend fortement du problème et des unités de tokenisation.

Exercice K — Coût combinatoire d'une recherche

Question. Une recherche possède 12 actions par état et explore jusqu'à profondeur 5. Nombre approximatif de feuilles sans pruning ?

Correction. $12^5 = 248\,832$ feuilles, auxquels s'ajoutent les nœuds intermédiaires. La mémorisation d'états déjà vus et l'élimination de mouvements inverses peuvent réduire le nombre de branches réellement explorées.

Exercice L — Q-learning

Question. $Q = 2$, $r = 1$, $\gamma = 0{,}9$, $\max Q' = 4$, $\alpha = 0{,}1$. Nouvelle valeur ?

Correction. La cible temporelle est $1 + 0{,}9 \times 4 = 4{,}6$; l'erreur est $4{,}6 - 2 = 2{,}6$. Mise à jour $Q_{new} = 2 + 0{,}1 \times 2{,}6 = 2{,}26$.

Exercice M — Architecture et usage

Question. Pourquoi ne faut-il pas appliquer manuellement softmax aux logits avant `nn.CrossEntropyLoss` dans PyTorch ?

Correction. Cette fonction attend des **logits bruts** et combine numériquement `log_softmax` et negative log-likelihood de manière stable. Lui donner déjà des probabilités transforme l'entrée, change la fonction optimisée et peut dégrader l'apprentissage. Le piège est vérifiable en lisant [R17] et la référence officielle `nn.CrossEntropyLoss`.

Exercice N — Recherche de source fiable

Question. Un assistant documentaire produit une phrase exacte, mais cite un passage qui ne la contient pas. Est-ce un échec de la réponse ?

Correction. C'est au minimum un échec de **traçabilité/citation**. La réponse peut être vraie pour d'autres raisons, mais elle n'est pas justifiée par la source indiquée. On doit donc séparer *answer correctness*, *citation support* et *retrieval completeness* dans les métriques.

Exercice O — Différence entre projet et preuve scientifique

Question. Ton Rubik's Cube solveur a résolu 20 mélanges de 3 mouvements. Peut-on écrire « il résout n'importe quel cube 3×3 » ?

Correction. Non. Le domaine testé se limite à une famille restreinte de configurations. Il faut documenter profondeurs, méthode de tirage, états effectivement atteints, temps limites et taux de réussite. L'assertion devrait être : « sur ces 20 mélanges de profondeur 3 générés selon cette méthode, 20 ont été résolus ».

28. Fiches de synthèse : ce que tu dois savoir expliquer sans notes

Notion	Niveau minimal attendu	Expérience discriminante
Produit matriciel	Calculer les dimensions de sortie	Provoquer une erreur de dimension et la corriger
Gradient	Relier dérivée, sens de variation, pas	Faire diverger une parabole avec un grand pas
Bayes	Distinguer sensibilité et valeur prédictive	Varier la prévalence
Modèle supervisé	Définir X, y, paramètres, loss	Entraîner une baseline
Généralisation	Justifier train/validation/test	Observer l'effet du leakage
Précision / rappel	Calculer à partir des TP/FP/FN	Déplacer un seuil
Réseau neuronal	Compter paramètres et activations	Supprimer la non-linéarité
Backprop	Expliquer la chaîne de gradients	Inspecter `.grad` dans PyTorch
PyTorch	Suivre forward/loss/backward/step	Retirer `zero_grad()` et observer
TF–IDF	Expliquer la pondération des mots	Comparer unigrammes et bigrammes

Notion	Niveau minimal attendu	Expérience discriminante
Embeddings	Expliquer espace vectoriel et contexte	Mesurer cosinus sur petits vecteurs
Attention	Expliquer QK^T, softmax et V	Faire une attention à deux tokens
LLM	Décrire le prochain-token et sampling	Générer avec un bigram
A* / IDA*	Justifier coûts, heuristique et mémoire	Comparer à BFS sur graphe simple
RL	Décrire politique, récompense, valeur	Effectuer une mise à jour Q-learning
Fiabilité	Distinguer score et déploiement	Simuler drift ou fuite

29. Sources complètes et ordre de lecture

Méthode de référencement. Les identifiants [Rxx] apparaissent au début de chaque chapitre et dans les légendes des figures. Les URLs renvoient aux pages **officielles** de l'éditeur, des auteurs ou des projets. Les ressources payantes ne sont pas copiées dans l'archive. Les chapitres de ce document sont rédigés spécifiquement pour ton parcours et reformulent les notions ; les figures sont originales. Vérification des pages de référence : octobre 2026.

[R01] PRIORITÉ 1 — James, Witten, Hastie, Tibshirani et Taylor. *An Introduction to Statistical Learning, with Applications in Python*, Springer (2023). Page officielle avec PDF en accès libre, labs et ressources : https://www.statlearning.com/. **Lire en premier :** introduction, régression, classification, validation. Puis arbres et SVM pour le projet Fraude.

[R02] PRIORITÉ 1 — Deisenroth, Faisal et Ong. *Mathematics for Machine Learning*, Cambridge University Press (2020). Site officiel des auteurs avec PDF disponible gratuitement : https://mml-book.github.io/. **Cibler :** algèbre linéaire, calcul vectoriel, probabilités et optimisation ; il n'est pas nécessaire de lire tous les chapitres d'affilée.

[R03] PRIORITÉ 1 — Zhang, Lipton, Li et Smola. *Dive into Deep Learning*, livre ouvert, mathématiques + notebooks : https://d2l.ai/. **Cibler :** régression, MLP, backpropagation, attention. L'ouvrage est exécutable et convient à une progression théorie–code.

[R04] PRIORITÉ 1 POUR NLP/LLM — Jurafsky et Martin. *Speech and Language Processing* (3e éd., manuscrit en évolution, diffusion du 19 août 2026). https://web.stanford.edu/~jurafsky/slp3/. **Cibler :** introduction, tokens, n-grams, classification et Transformers. Le manuscrit n'est pas une édition finale stabilisée ; vérifie toujours le chapitre de la version consultée.

[R05] PRIORITÉ 2 — Simon J. D. Prince. *Understanding Deep Learning* (2023). Site de l'auteur : https://udlbook.com/. **Utiliser :** explications détaillées de l'optimisation, architectures et théorie après une première approche [R03]. Ne pas tenter de lire l'intégralité en quatre semaines.

[R06] PRIORITÉ 1 POUR RUBIK — UC Berkeley. *CS188: Introduction to Artificial Intelligence*, supports de cours (Fall 2026) : https://inst.eecs.berkeley.edu/~cs188/fa26/. **Cibler :** recherche non informée, A*, heuristiques, MDP. Supports de cours, pas certification autonome.

[R07] PRIORITÉ 2 — Russell et Norvig. *Artificial Intelligence: A Modern Approach*, Pearson, 4e édition

(livre commercial). https://aima.cs.berkeley.edu/. **Utiliser :** cadre général de recherche, agents et planification ; source de fond, pas achat obligatoire.

[R08] PRIORITÉ 3 — Stanford. *CS336: Language Modeling from Scratch*, Spring 2026 : https://cs336.stanford.edu/. **À réserver :** phase LLM avancée après PyTorch et attention. Supports et exercices de niveau universitaire exigeant.

[R09] PRIORITÉ 1 POUR PYTHON SCIENTIFIQUE — NumPy. Guide officiel débutant : https://numpy.org/doc/stable/user/absolute_beginners.html.

[R10] PRIORITÉ 1 — Google. *Machine Learning Crash Course*, cours gratuit sur régression, classification et descente de gradient : https://developers.google.com/machine-learning/crash-course.

[R11] PRIORITÉ 1 POUR MÉTRIQUES — scikit-learn. *Model Evaluation*: https://scikit-learn.org/stable/modules/model_evaluation.html.

[R12] PRIORITÉ 1 — pandas. *Getting started tutorials*: https://pandas.pydata.org/docs/getting_started/intro_tutorials/.

[R13] PRIORITÉ 2 — Matplotlib. Documentation et galeries officielles : https://matplotlib.org/stable/tutorials/index.html.

[R14] PRIORITÉ 1 POUR MODÈLES CLASSIQUES — scikit-learn. Documentation officielle : https://scikit-learn.org/stable/supervised_learning.html.

[R15] PRIORITÉ 1 POUR LE PROTOCOLE — scikit-learn. *Common pitfalls and recommended practices* : https://scikit-learn.org/stable/common_pitfalls.html.

[R16] PRIORITÉ 2 — scikit-learn. *Precision-Recall*: https://scikit-learn.org/stable/auto_examples/model_selection/plot_precision_recall.html.

[R17] PRIORITÉ 1 POUR DEEP LEARNING PRATIQUE — PyTorch. *Learn the Basics* : https://docs.pytorch.org/tutorials/beginner/basics/intro.html.

[R18] PRIORITÉ 1 POUR NLP CLASSIQUE — scikit-learn. *Working with Text Data* : https://scikit-learn.org/stable/tutorial/text_analytics/working_with_text_data.html.

[R19] PRIORITÉ 1 POUR OUTILS NLP/LLM — Hugging Face. *LLM Course* : https://huggingface.co/learn/llm-course/en/chapter1/1. Familiarité préalable avec Python nécessaire.

[R20] ARTICLE FONDATEUR — Vaswani et al. *Attention Is All You Need*, NeurIPS 2017 : https://arxiv.org/abs/1706.03762. Lire sections 3.1–3.2 après les chapitres sur matrices et softmax.

[R21] RÉFÉRENCE TECHNIQUE CUBE — Herbert Kociemba. *The Two-Phase Algorithm* : https://www.kociemba.org/twophase.htm.

[R22] PRIORITÉ 2 POUR RL — Sutton et Barto. *Reinforcement Learning: An Introduction*, 2e édition (2018), page académique : http://incompleteideas.net/book/the-book-2nd.html.

[R23] PRIORITÉ 3 — Hugging Face. *Deep Reinforcement Learning Course* : https://huggingface.co/learn/deep-rl-course/unit0/introduction.

[R24] PRIORITÉ 1 POUR RISQUES — NIST. *AI Risk Management Framework 1.0* et ressources : https://www.nist.gov/itl/ai-risk-management-framework. Ce document est un cadre volontaire et peut connaître des révisions ; vérifier la version consultée.

[R25] PRIORITÉ 2 POUR SÉCURITÉ LLM — OWASP. *OWASP Top 10 for LLM Applications* : https://genai.owasp.org/llm-top-10/. Document de risque, non certificat de conformité.

[R26] PRIORITÉ 2 POUR VISION — Stanford. *CS231n: Deep Learning for Computer Vision* : https:

//cs231n.stanford.edu/.

[R27] PRIORITÉ 2 POUR ORGANISATION — Schwaber et Sutherland. *The Scrum Guide* (édition officielle) : https://scrumguides.org/. Cadre de processus, pas unique bonne manière de travailler.

[R28] FACULTATIF — Camille Fournier. *The Manager's Path* (O'Reilly, ouvrage commercial, 2017) : https://www.oreilly.com/library/view/the-managers-path/9781491973882/.

30. Feuille de lecture progressive : quoi ouvrir, et quand ?

Moment	Une seule source prioritaire	Complément éventuel	Production attendue
Mois 1, S1	[R02] notions mathématiques ciblées	[R09, R12]	Vecteurs, matrices, stats en code
Mois 1, S2	[R01] régression/classification	[R10]	Descente de gradient sans ML library
Mois 1, S3	[R03] perceptrons et backprop	[R17]	MLP PyTorch entraîné
Mois 1, S4	[R11] métriques + [R04] tokens	[R19]	Fiche NLP/évaluation
Mois 2, Fraude	[R01] classification/arbres	[R15, R11]	Baseline, seuils, bilan d'erreurs
Mois 3, Avis	[R18] TF–IDF et classification	[R04]	Catégorie + sentiment
Mois 4, Cube	[R06] graphes, A*, heuristiques	[R21]	Solveur court + benchmark
Mois 5 optionnel	[R04] Transformers + [R19]	[R20, R08]	Petit LM et évaluation
Grand défi	Source technique du domaine choisie	[R24, R25, R26]	Revue de littérature + MVP mesuré

Règle pratique : ne poursuis pas simultanément cinq ressources. Prends le **cours français présent**, la **source P1** du chapitre et, si une formule résiste, une ressource complémentaire. Note systématiquement ce que tu comprends *grâce* à la source et ce qui reste à tester.